Université Cadi Ayad

École Nationale des Sciences Appliquées de Marrakech

Code212 de l'UCA

Rapport de Projet de Fin d'Année

Mise en place d'une plateforme de gestion des accès juste-à-temps (Just-In-Time Access)

Realisé par : ISEDRHAS Aya

Sous la supervision de : M. Walid Bouarifi

Stage effectué à : Centre Code212 de l'UCA

Période de stage : du 17/07/2026 au 31/08/2026

Année universitaire 2025–2026

Dédicace

À mes parents, pour leur amour inconditionnel, leur soutien et leurs encouragements continus.

À ma famille, pour sa sa confiance et son accompagnement tout au long de mon parcours.

À toutes les personnes qui me soutiennent, encouragent et inspirent durant mes études.

Je dédie humblement ce travail à chacun d'entre vous.

Remerciements

Je tiens à remercier **M. Walid Bouarifi** du fond de mon cœur de m'avoir octroyé l'opportunité de passer mon stage PFA au sein du centre code212 de l'UCA. Ce fut une très belle expérience très enrichissante et formatrice, qui m'a permis de développer de nouvelles compétences en terme de gestion des accès et des identités (IAM).

Mes remerciements s'adressent également à l'ensemble du corps enseignant de l'ENSA Marrakech pour leurs efforts remarquables et leur soutien, leur disponibilité et leur engagement ont largement contribué à ma formation et à la réalisation de ce travail.

Je n'oublierai pas également ma famille pour leur patience, leur compréhension et leurs encouragements continus durant cette période de travail intense.

Acknowledgments

I would like to express my sincere gratitude to **Mr. Walid Bouarifi** for giving me the opportunity to carry out my End-of-Year Project internship at the Code 212 Center of Cadi Ayyad University (UCA). This internship was a highly enriching and valuable experience that enabled me to acquire new knowledge and develop practical skills, particularly in the field of Identity and Access Management (IAM).

I would also like to extend my sincere thanks to the entire teaching staff of ENSA Marrakech for their remarkable efforts throughout my academic journey. Their guidance, availability, support, and dedication have greatly contributed to my training and to the successful completion of this work.

Finally, I am deeply grateful to my family for their patience, understanding, continuous encouragement, and unwavering support throughout this demanding period.

Résumé

Le présent rapport s'inscrit dans le cadre d'un stage PFA réalisé au sein du Centre Code212 de l'Université Cadi Ayyad (UCA). Il porte sur la conception et la mise en œuvre d'une solution de gestion des accès privilégiés Just-In-Time (JIT) destinée à automatiser l'octroi d'autorisations temporaires à des ressources cloud. Le projet vise à permettre à un utilisateur d'obtenir, pour une durée limitée et après plusieurs contrôles de sécurité, les privilèges nécessaires à l'exécution d'une tâche sur AWS, sans disposer en permanence de droits privilégiés sur l'infrastructure.

Cette approche répond à la problématique des privilèges permanents, ou *standing privileges*, qui augmentent la surface d'attaque lorsqu'ils restent actifs en dehors d'un besoin réel. La solution applique ainsi le principe de *Zero Standing Privilege*, selon lequel les droits sensibles ne sont accordés qu'à la demande, après justification et validation, et pour une durée limitée.

Le processus repose sur plusieurs composants complémentaires. L'utilisateur s'authentifie via Keycloak puis soumet une demande précisant le rôle souhaité, la durée d'accès et un ticket Jira servant de justification. Open Policy Agent (OPA) analyse ensuite la requête à partir de politiques écrites en Rego selon une approche *Policy as Code* et un modèle ABAC, en vérifiant notamment le rôle de l'utilisateur, les caractéristiques du ticket, la durée demandée et les contraintes définies par la politique de sécurité.

Lorsque la demande est autorisée, une authentification multifacteur basée sur TOTP est exigée avant l'octroi des privilèges. Après validation, le backend utilise AWS Security Token Service (STS) pour assumer un rôle IAM prédéfini et générer des identifiants temporaires limités dans le temps, évitant ainsi l'attribution de comptes ou d'identifiants AWS permanents aux utilisateurs finaux.

Afin de sécuriser leur distribution, ces identifiants ne sont pas affichés directement dans l'interface web. Ils sont récupérés par un agent local via une communication HTTPS/TLS, puis injectés dans une session PowerShell dédiée permettant l'utilisation de l'AWS CLI avec les permissions du rôle temporaire obtenu. Une fois la durée de validité expirée, les accès deviennent automatiquement inutilisables.

Enfin, Splunk centralise les principaux événements de sécurité générés par la plateforme, notamment les demandes d'accès, les refus de politiques, les échecs de MFA et les octrois temporaires, afin d'assurer la traçabilité, l'audit et la supervision du système.

Mots-clés : Just-In-Time Access, Zero Standing Privilege, IAM, Keycloak, OpenID Connect, Open Policy Agent, Policy as Code, ABAC, MFA, TOTP, AWS STS, AssumeRole, IAM Role, AWS, Jira, Splunk, gestion des accès privilégiés.

Abstract

This report was prepared as part of an End-of-Year Project internship (PFA) carried out at the Code 212 Center of Cadi Ayyad University (UCA). It focuses on the design and implementation of a Just-In-Time (JIT) privileged access management solution intended to automate the granting of temporary permissions to cloud resources. The project aims to allow a user to obtain, for a limited period and after several security checks, the privileges required to perform a task on AWS, without permanently holding privileged permissions within the infrastructure.

This approach addresses the issue of standing privileges, which increase the attack surface when they remain active beyond an actual operational need. The proposed solution therefore applies the Zero Standing Privilege principle, according to which sensitive permissions are granted only on demand, after justification and validation, and for a limited duration.

The workflow relies on several complementary components. The user authenticates through Keycloak and then submits an access request specifying the requested role, the access duration, and a Jira ticket used as justification. Open Policy Agent (OPA) subsequently evaluates the request through policies written in Rego, following a Policy as Code approach and an ABAC model, by checking several conditions such as the user's role, ticket attributes, requested duration, and the constraints defined by the security policy.

When the request is authorized, a TOTP-based multi-factor authentication step is required before privileges are granted. Once validated, the backend uses AWS Security Token Service (STS) to assume a predefined IAM role and generate time-limited temporary credentials, thereby avoiding the use of permanent AWS accounts or long-lived credentials for end users.

To secure their distribution, these credentials are not displayed directly in the web interface. They are retrieved by a local agent through an HTTPS/TLS-protected communication channel and injected into a dedicated PowerShell session, allowing the user to operate the AWS CLI with the permissions associated with the temporary role. Once the validity period expires, the granted access automatically becomes unusable.

Finally, Splunk centralizes the main security events generated by the platform, including access requests, policy denials, MFA failures, and temporary access grants, in order to provide traceability, auditing, and system monitoring.

Keywords : Just-In-Time Access, Zero Standing Privilege, IAM, Keycloak, OpenID Connect, Open Policy Agent, Policy as Code, ABAC, MFA, TOTP, AWS STS, AssumeRole, IAM Role, AWS, Jira, Splunk, privileged access management.

Table des matières

Dédicace ii

Remerciements iii

Acknowledgments iv

Résumé v

Abstract vi

Introduction générale 1

1 Contexte général 4

Introduction . . . 4

1.1 Présentation de l'organisme d'accueil . . . 4

1.1.1 Présentation générale de Code212 . . . 4

1.1.2 Philosophie et principes du centre . . . 5

1.1.3 Missions de Code212-UCA . . . 5

1.1.4 Approche pédagogique . . . 6

1.1.5 Domaines de formation . . . 7

1.1.6 Formations et certifications techniques . . . 8

1.1.7 Infrastructures et espaces d'innovation . . . 8

1.1.8 Écosystème, clubs et partenariats . . . 9

1.1.9 Vision de Code212-UCA . . . 9

1.1.10 Code212-UCA comme cadre du Projet de Fin d'Année . . . 10

1.2 Cadre Institutionnel . . . 10

1.2.1 École Nationale des Sciences Appliquées de Marrakech . . . 10

1.2.2 Filière GCDSTE . . . 11

1.3 Évolution de la Gestion des Identités et des Accès . . . 11

1.3.1 Historique et tendances . . . 11

1.3.2 Risques liés aux privilèges permanents . . . 11

1.3.3 Émergence du Just-In-Time Access . . . 12

1.3.4 Lien avec le projet . . . 12

1.4 Problématique et Enjeux . . . 12

1.4.1 Limites des approches traditionnelles . . . 12

1.4.2 Défis liés à la gestion dynamique des accès . . . 13

1.4.3 Enjeux du projet . . . 13

1.5 Approche Proposée . . . 13
1.5.1 Architecture fonctionnelle . . . 13
1.6 Périmètre et Contraintes . . . 14
1.6.1 Périmètre fonctionnel . . . 14
1.6.2 Éléments exclus du périmètre . . . 14
1.6.3 Contraintes identifiées . . . 15
Conclusion . . . 15

2 Cadre Théorique et État de l'Art 17
Introduction . . . 17
2.1 Gestion des identités et des accès (IAM) . . . 17
2.1.1 Identity and Access Management : définitions générales . . . 17
2.1.2 Authentification versus autorisation . . . 18
2.1.3 Le problème des privilèges permanents . . . 19
2.1.4 Le principe de Zero Standing Privilege . . . 19
2.1.5 Just-In-Time Access . . . 20
2.1.6 Le principe du moindre privilège . . . 20
2.2 Protocoles d'authentification et d'autorisation . . . 21
2.2.1 OAuth 2.0 . . . 21
2.2.2 OpenID Connect . . . 22
2.2.3 Le flow Authorization Code avec PKCE . . . 22
2.2.4 JSON Web Token et JWKS . . . 23
2.3 Modèles théoriques de contrôle d'accès . . . 24
2.3.1 Role-Based Access Control (RBAC) . . . 24
2.3.2 Attribute-Based Access Control (ABAC) . . . 24
2.3.3 Policy-as-Code et Open Policy Agent . . . 25
2.4 Concepts cloud et gestion des identités machine . . . 26
2.4.1 Identités permanentes versus identités fédérées . . . 26
2.4.2 Le mécanisme AssumeRole . . . 27
2.4.3 Expiration cryptographique des credentials . . . 27
2.5 Observabilité et sécurité opérationnelle . . . 28
2.5.1 Security Information and Event Management (SIEM) . . . 28
2.5.2 Le principe de non-interférence de la couche d'observabilité . . . 28
2.5.3 Piste d'audit . . . 28
2.6 Positionnement par rapport à l'état de l'art . . . 29
Conclusion . . . 29

3 Technologies Adoptées 30
Introduction . . . 30
3.1 Keycloak — Gestion des identités et des accès . . . 30

3.2 Open Policy Agent et Rego — Moteur de décision 31
3.3 Jira — Système de ticketing 31
3.4 Python et FastAPI — Backend applicatif 32
3.5 AWS IAM, STS et boto3 — Gestion des accès temporaires 32
3.6 TOTP et pyotp — Authentification renforcée 33
3.7 Splunk — Journalisation et supervision 34
3.8 Docker et Docker Compose — Conteneurisation 35
3.9 HTML, CSS et JavaScript — Interface utilisateur 35
3.10 PowerShell et agent local — Distribution des accès 36
3.11 TLS et certificats auto-signés — Sécurité du transport 37
Conclusion 37

4 Conception et Fonctionnement de plateforme de gestion des accès Just-In-Time 39
Introduction 39
4.1 Architecture globale de la solution 39
4.2 Déroulement global du workflow 40
4.3 Mise en œuvre des composants techniques 40
4.3.1 Configuration de Keycloak 41
4.3.2 Mise en place d'Open Policy Agent (OPA) et des politiques Rego 44
4.3.3 Intégration avec Jira 47
4.3.4 Authentification multifacteur par TOTP 51
4.4 Mise en œuvre des accès temporaires avec AWS IAM et STS 53
4.4.1 Création de l'utilisateur IAM applicatif 53
4.4.2 Autorisation d'assumer les rôles 54
4.4.3 Création des rôles IAM 55
4.4.4 Configuration de la relation de confiance 56
4.4.5 Création des politiques de permissions S3 56
4.4.6 Création de la ressource Amazon S3 58
4.4.7 Fonctionnement de AWS STS dans le projet 58
4.5 Agent local et distribution sécurisée des accès temporaires 58
4.5.1 Principe de distribution 59
4.5.2 Ouverture d'une session PowerShell dédiée 59
4.5.3 Utilisation d'un poste de travail légitime 59
4.5.4 Expiration automatique de la session 60
4.5.5 Apport de l'agent local 60
4.6 Journalisation et supervision avec Splunk 60
Conclusion 62

5 Architecture et Implémentation de l'Application Just-in-Time Access 63
Introduction . . . 63
5.1 Architecture du projet . . . 63
5.2 Interface utilisateur . . . 64
5.2.1 `index.html` . . . 64
5.2.2 `style.css` . . . 65
5.2.3 `config.js` . . . 65
5.2.4 `app.js` . . . 65
5.3 Backend et orchestration du workflow . . . 65
5.3.1 `main.py` . . . 65
5.3.2 `auth.py` . . . 66
5.3.3 `models.py` . . . 66
5.3.4 `session_store.py` . . . 66
5.4 Services d'intégration . . . 66
5.4.1 `jira_service.py` . . . 66
5.4.2 `opa_service.py` . . . 66
5.4.3 `mfa_service.py` . . . 66
5.4.4 `aws_service.py` . . . 67
5.4.5 `splunk_service.py` . . . 67
5.5 Politique d'autorisation OPA . . . 67
5.6 Agent local . . . 67
5.7 Sécurisation des communications . . . 68
5.8 Infrastructure Docker . . . 68
5.9 Scripts de démarrage . . . 68
5.10 Implémentation de l'application Just-In-Time Access . . . 69
5.10.1 Page d'accueil et authentification . . . 69
5.10.2 Soumission d'une demande d'accès . . . 70
5.10.3 Refus d'une demande non conforme . . . 70
5.10.4 Authentification multifacteur . . . 71
5.10.5 Octroi de l'accès et lancement de la session . . . 72
5.10.6 Session PowerShell et utilisation de l'AWS CLI . . . 73
5.10.7 Vérification du résultat côté AWS . . . 74
5.10.8 Journalisation et supervision avec Splunk . . . 75
Conclusion . . . 76

Conclusion générale 78

Bibliographie 81

Table des figures

1.1 Logo du Centre Code212 4
1.2 Logo de ENSA Marrakech 10

2.1 Étapes principales d'un processus IAM : identification, authentification et autorisation 18
2.2 Flux d'authentification entre l'utilisateur , l'application et le fournisseur d'identité 18
2.3 Authentification VS autorisation 19
2.4 Comparaison entre le modèle de privilèges permanents (standing privileges) et le modèle Zero Standing Privilege 20
2.5 Principe du Just-In-Time Access 20
2.6 Le principe du moindre privilège 21
2.7 Flux du protocole OpenID Connect 22
2.8 Séquence du flow Authorization Code avec PKCE 23
2.9 Vérification de la signature du JSON Web Token via JWKS 24
2.10 Comparaison structurelle entre les modèles RBAC et ABAC 25
2.11 Principe de fonctionnement d'Open Policy Agent dans une architecture policy- as-code 26
2.12 Principe général du mécanisme d'endossement temporaire de rôle (AssumeRole) 27

3.1 Logo de Keycloak 30
3.2 Logo de Open Policy Agent 31
3.3 Logo de Jira 31
3.4 Logos de Python et FastAPI 32
3.5 Logo de Amazon Web Services 32
3.6 Microsoft Authenticator 33
3.7 Logo de Splunk 34
3.8 Logo de Docker 35
3.9 Technologies utilisées pour l'interface Web 35
3.10 Logo de PowerShell 36
3.11 Sécurisation des communications par TLS 37

4.1 Architecture globale de la plateforme de gestion des accès temporaires Just In Time 40
4.2 Création du realm `jit-realm` dans la console d'administration Keycloak 41

4.3 Clients `jit-frontend` et `jit-backend` configurés dans Keycloak 42
4.4 Rôles métier définis dans le realm `jit-realm` 43
4.5 Conteneur OPA actif, exposant le port 8181 44
4.6 domaine Atlassian 48
4.7 Espace Jira `JIT-Access` dédié à la gestion des tickets de demande d'accès temporaire 48
4.8 Clé API de Jira 49
4.9 Utilisateur IAM dédié à l'application JIT 54
4.10 Politique d'autorisation associée à l'utilisateur IAM 54
4.11 Politique d'autorisation détaillée associée à l'utilisateur IAM 55
4.12 Rôles IAM utilisés pour les accès temporaires 55
4.13 Relation de confiance du rôle `s3ReadRole` 56
4.14 Politiques IAM utilisées pour les accès S3 56
4.15 Permissions de lecture définies par `s3ReadPolicy` 57
4.16 Permissions d'écriture définies par `s3WritePolicy` 57
4.17 Association de la politique `s3ReadPolicy` au rôle `s3ReadRole` 57
4.18 Bucket Amazon S3 utilisé pour la validation des accès temporaires 58

5.1 Page d'accueil du portail de demande d'accès 69
5.2 Formulaire de soumission d'une demande d'accès temporaire 70
5.3 Écran affiché lorsque la demande est refusée par la politique de sécurité . 71
5.4 Étape d'authentification multifacteur – enrôlement TOTP 72
5.5 Confirmation de l'octroi de l'accès et ouverture automatique de la session 73
5.6 Session PowerShell disposant des identifiants temporaires et exécution de commandes AWS CLI 74
5.7 Fichier déposé dans le bucket S3 grâce aux privilèges temporaires obtenus 74
5.8 Événement `access_denied` – demande rejetée par la politique OPA (ticket JIT-1) 75
5.9 Événement `mfa_required` – demande autorisée, en attente du code TOTP 75
5.10 Événement `mfa_failed` – code TOTP invalide soumis par l'utilisateur . . 75
5.11 Événement `access_granted` – provisioning AWS STS réussi (rôle `s3-write-access`, 40 minutes) 76
5.12 Événement `agent_credentials_claimed` – retrait des identifiants par l'agent local 76

Liste des tableaux

3.1 Synthèse des technologies adoptées 38

4.1 Configuration des deux clients Keycloak du projet 42
4.2 Matrice de correspondance entre rôles métier et rôles temporaires 46
4.3 Catalogue des événements de sécurité transmis à Splunk 62

Introduction générale

La transformation numérique des organisations s'accompagne d'une utilisation croissante des services cloud et d'une multiplication des accès aux ressources sensibles. Dans ce contexte, la gestion des identités et des privilèges constitue un enjeu majeur de cybersécurité. Les mécanismes classiques reposant sur des droits permanents accordés aux utilisateurs peuvent exposer les systèmes à des risques importants en cas de compromission d'un compte. Il devient donc nécessaire d'adopter des approches capables de limiter les privilèges dans le temps, de renforcer les contrôles d'accès et d'assurer une traçabilité complète des opérations réalisées.

Problématique

Dans de nombreuses architectures d'entreprise, certains utilisateurs disposent de privilèges élevés de manière permanente alors que ces autorisations ne sont nécessaires que pour des opérations ponctuelles. Ces *standing privileges* augmentent considérablement la surface d'attaque, puisqu'un compte compromis peut permettre un accès immédiat à des ressources sensibles.

À cela s'ajoute la difficulté de vérifier systématiquement la légitimité d'une demande d'accès, son contexte, sa durée ainsi que les autorisations réellement nécessaires. Une gestion manuelle de ces accès peut également engendrer des erreurs humaines, un manque de traçabilité et des délais importants dans l'attribution ou la révocation des privilèges.

La problématique principale de ce projet consiste donc à répondre à la question suivante :

> *Comment concevoir et mettre en place une solution automatisée permettant d'accorder des privilèges cloud uniquement lorsqu'ils sont nécessaires, pour une durée limitée, après vérification de la légitimité de la demande et de l'identité de l'utilisateur, tout en assurant la traçabilité des différentes opérations réalisées ?*

Pour répondre à cette problématique, le projet s'appuie sur le principe de *Just-In-Time Access*, associé à l'approche *Zero Standing Privilege*. L'objectif est de remplacer autant que possible les privilèges permanents par des autorisations temporaires, attribuées dynamiquement en fonction du contexte de la demande et automatiquement invalidées après expiration.

Objectif du projet

L'objectif principal de ce projet est de concevoir et d'implémenter une plateforme de gestion des accès privilégiés *Just-In-Time* permettant d'automatiser le processus d'attribution de droits temporaires sur des ressources cloud AWS.

La solution développée doit prendre en charge l'ensemble du cycle de demande d'accès, depuis l'authentification de l'utilisateur jusqu'à l'obtention des identifiants temporaires. Elle doit notamment permettre de vérifier l'identité de l'utilisateur à travers **Keycloak**, de contrôler la justification de la demande à partir d'un ticket **Jira**, puis d'évaluer les conditions d'autorisation à travers **Open Policy Agent (OPA)** selon une approche *Policy as Code*.

Lorsque les règles de sécurité sont satisfaites, une authentification multifacteur basée sur un code **TOTP** est imposée avant l'attribution effective des privilèges. Le backend utilise ensuite **AWS Security Token Service (STS)** afin d'assumer un rôle IAM prédéfini et de générer des identifiants temporaires ayant une durée de validité limitée.

Le projet vise également à sécuriser la transmission de ces identifiants. Pour cela, un agent local est utilisé afin de récupérer les informations nécessaires via une communication protégée par **HTTPS/TLS**, puis de les injecter dans une session PowerShell dédiée. Cette approche évite l'affichage direct des identifiants AWS dans l'interface utilisateur et permet de limiter leur exposition.

Enfin, un mécanisme de supervision basé sur **Splunk** est intégré afin de centraliser les événements de sécurité générés tout au long du processus, notamment les demandes d'accès, les décisions prises par OPA, les échecs d'authentification multifacteur et les octrois de privilèges temporaires.

Ainsi, le projet cherche à proposer une architecture combinant gestion des identités, contrôle d'accès dynamique, authentification renforcée, privilèges temporaires et supervision, tout en respectant le principe du moindre privilège.

Méthodologie adoptée

La démarche suivie dans ce projet est structurée en plusieurs phases successives permettant de progresser depuis l'étude du besoin jusqu'à la validation complète de la solution.

Phase 1 — Étude et conception

Cette première phase a consisté à analyser les problématiques liées aux privilèges permanents et à étudier les principes du *Just-In-Time Access*, du *Zero Standing Privilege* et de la gestion des identités et des accès. Elle a également permis de définir l'architecture

générale de la solution, les différents composants nécessaires ainsi que les interactions entre Keycloak, Jira, OPA, AWS STS, l'agent local et Splunk.

Phase 2 — Déploiement et intégration

La deuxième phase a porté sur la mise en place de l'environnement technique et l'intégration progressive des différents composants de l'architecture. Les services Keycloak, OPA et Splunk ont notamment été déployés dans un environnement conteneurisé, puis reliés au backend de l'application, tandis que la communication avec AWS STS et le mécanisme de récupération des identifiants temporaires ont été configurés.

Phase 3 — Tests et validation

Cette phase a été consacrée à la vérification du fonctionnement de la solution à travers plusieurs scénarios. Des tests positifs et négatifs ont été réalisés afin de valider l'authentification des utilisateurs, l'évaluation des politiques OPA, la vérification du MFA, la génération des identifiants temporaires AWS ainsi que leur utilisation à travers une session PowerShell disposant uniquement des privilèges accordés.

Phase 4 — Analyse et documentation

La dernière phase a consisté à analyser les résultats obtenus, à vérifier la cohérence globale du workflow et à évaluer les apports de l'architecture en matière de limitation et de traçabilité des privilèges. Elle comprend également la documentation des choix techniques, des différentes étapes de mise en œuvre et la rédaction du présent rapport.

Chapitre 1

Contexte général

Introduction

Dans le cadre de ce Projet de Fin d'Année, il est essentiel de présenter l'environnement académique et professionnel dans lequel le stage a été réalisé. Cette partie a ainsi pour objectif d'introduire les deux principales structures ayant encadré ce travail, à savoir l'École Nationale des Sciences Appliquées de Marrakech (ENSA Marrakech), l'établissement de formation, ainsi que le Centre Code 212 de l'Université Cadi Ayyad (UCA), organisme d'accueil du stage. "'latex

1.1 Présentation de l'organisme d'accueil

Le stage ayant donné lieu à ce Projet de Fin d'Année a été réalisé au sein du **Centre Code212 de l'Université Cadi Ayyad (UCA)**. Ce centre accompagne la transformation numérique de l'enseignement supérieur et vise à renforcer les compétences technologiques des étudiants afin de mieux répondre aux besoins du monde professionnel.

Code212 se présente comme un **Centre des Futures Compétences**, orienté vers l'acquisition de compétences numériques, l'innovation, l'apprentissage pratique et le rapprochement entre l'université et le monde professionnel.

Figure 1.1 – Logo du Centre Code212

1.1.1 Présentation générale de Code212

Le réseau national des centres **Code212** a pour objectif de développer les compétences numériques au sein des universités marocaines. Il met à disposition des étudiants des environnements modernes favorisant l'apprentissage de technologies en constante évolution.

Le Centre Code212 de l'Université Cadi Ayyad constitue ainsi un espace dédié au développement des compétences d'avenir. Il propose des formations complémentaires et encourage une pédagogie fondée sur la pratique, l'autonomie, la collaboration et la réalisation de projets.

Cette orientation répond à l'évolution rapide des métiers du numérique, notamment dans les domaines du Cloud Computing, de l'intelligence artificielle, de la cybersécurité, de la Data et du DevOps.

Code212 cherche ainsi à rapprocher la formation académique des compétences opérationnelles attendues dans le monde professionnel. Le centre s'adresse à des étudiants de différentes disciplines et favorise l'acquisition d'une **double compétence**, associant spécialité académique et compétences numériques complémentaires.

1.1.2 Philosophie et principes du centre

La philosophie de Code212-UCA repose principalement sur trois axes : **l'innovation**, **la collaboration** et **l'excellence**.

Innovation continue

L'innovation occupe une place centrale dans la démarche du centre. Les étudiants sont encouragés à expérimenter de nouvelles technologies et à transformer leurs idées en solutions concrètes.

Cette approche vise également à développer leur capacité d'adaptation face à l'évolution rapide des technologies et des besoins professionnels.

Collaboration et communauté

Code212 favorise l'apprentissage collectif à travers les échanges entre étudiants, le partage de connaissances et les projets collaboratifs.

Cette démarche contribue au développement de compétences telles que le travail en équipe, la communication et la résolution collective de problèmes.

Excellence et performance

Le centre encourage une culture de l'excellence à travers ses formations, certifications et projets. L'objectif est de permettre aux étudiants de développer des compétences pratiques directement valorisables dans leur parcours professionnel.

1.1.3 Missions de Code212-UCA

Les principales missions du Centre Code212-UCA concernent le développement des compétences, la transformation numérique et l'amélioration de l'employabilité des

étudiants.

Développement et certification des compétences

Le centre propose des parcours de formation et des certifications permettant aux étudiants d'acquérir des compétences complémentaires à leur cursus universitaire.

Ces certifications permettent de valoriser des compétences spécifiques et de mieux adapter les profils des étudiants aux besoins du marché du travail.

Accompagnement de la transformation numérique

Code212 contribue à la transformation numérique de l'université en favorisant l'apprentissage des technologies émergentes.

Les domaines concernés incluent notamment l'intelligence artificielle, la cybersécurité, le Cloud Computing, la Data Science, le développement logiciel, l'Internet des Objets et les systèmes embarqués.

Renforcement de l'employabilité

Le centre vise également à préparer les étudiants à leur insertion professionnelle en combinant compétences techniques et compétences transversales.

Les activités proposées permettent ainsi de compléter la formation académique par des expériences pratiques proches des environnements professionnels.

Développement de projets à impact

Code212 encourage la réalisation de projets technologiques visant à répondre à des problématiques concrètes.

Le présent Projet de Fin d'Année s'inscrit dans cette orientation puisqu'il traite d'une problématique actuelle de cybersécurité liée à la gestion dynamique des identités et des accès.

1.1.4 Approche pédagogique

La démarche pédagogique de Code212 place l'étudiant au centre de son apprentissage et privilégie les approches pratiques et collaboratives.

Apprentissage par la pratique

Le centre s'appuie notamment sur le principe du *Learning by Doing*. Les étudiants sont amenés à manipuler les technologies étudiées, résoudre des problèmes et développer des projets.

Cette méthode permet de consolider les connaissances théoriques tout en développant l'autonomie. Le *Peer Learning* complète cette approche par l'apprentissage entre étudiants.

Blended Learning

Code212 adopte également une approche de type *Blended Learning*, associant apprentissage en ligne, activités pratiques et séances en présentiel.

Cette combinaison permet d'offrir davantage de flexibilité tout en conservant une dimension pratique et un accompagnement pédagogique.

Développement des compétences transversales

Le centre accorde également une importance particulière aux *soft skills*, notamment la communication, la créativité, l'esprit critique, la coopération et la résolution de problèmes.

Ces compétences complètent les connaissances techniques et facilitent l'intégration dans des environnements professionnels multidisciplinaires.

1.1.5 Domaines de formation

Le Centre Code212-UCA propose un catalogue de formations couvrant plusieurs disciplines afin de permettre aux étudiants de développer des compétences complémentaires à leur formation principale.

Technologies numériques

Les principales formations dans le domaine numérique concernent notamment :

— le développement Web et mobile ;
— l'intelligence artificielle et le Machine Learning ;
— la cybersécurité ;
— le Cloud Computing et le DevOps ;
— l'Internet des Objets ;
— l'ingénierie logicielle ;
— la Data Science et le Big Data ;
— la Data Engineering ;
— la Blockchain et le Web3.

Ces domaines correspondent à des secteurs technologiques fortement présents dans les environnements professionnels actuels.

Ingénierie et sciences

Code212 propose également des formations liées à la robotique, aux systèmes embarqués, à la modélisation numérique, à l'analyse de données et à la géomatique.

Ces domaines permettent d'appliquer les technologies numériques à des problématiques scientifiques et techniques variées.

Sciences humaines, juridiques et économiques

Les formations ne sont pas limitées aux spécialités techniques. Le centre propose également des thématiques liées aux humanités numériques, au droit du numérique, à l'économie numérique, à la FinTech, à la gestion de projet et à la communication.

Cette diversité traduit la volonté de faire du numérique une compétence transversale accessible à plusieurs disciplines universitaires.

1.1.6 Formations et certifications techniques

Le catalogue de Code212-UCA comprend plusieurs niveaux de formation, allant des notions fondamentales jusqu'à des sujets avancés.

Les parcours couvrent notamment le développement Web, les bases de données, le développement Full Stack, les API, les tests logiciels, la sécurité Web, le DevOps et les applications Cloud.

Cette progression permet aux apprenants d'acquérir une vision globale du cycle de vie d'une application, depuis son développement jusqu'à sa sécurisation, son déploiement et sa supervision.

1.1.7 Infrastructures et espaces d'innovation

Code212-UCA se présente comme un espace *phygital* combinant ressources physiques et numériques. Ses différents espaces sont conçus pour favoriser l'expérimentation, la créativité et le développement de projets.

Espace Coding

L'espace **Coding** est dédié à la programmation et aux projets numériques. Il permet notamment de travailler sur des activités liées au développement, à la cybersécurité et à l'Internet des Objets.

Espace Talks

L'espace **Talks** accueille des conférences, présentations, échanges et sessions de partage autour de différentes thématiques.

Il favorise ainsi la communication et les échanges entre étudiants, enseignants et professionnels.

Espace Robotic

L'espace **Robotic** est consacré à la conception et au prototypage de systèmes robotiques et d'objets connectés.

Il permet aux étudiants de mettre en pratique des compétences liées à la programmation, à l'électronique et aux systèmes embarqués.

Data Lab

Le **Data Lab** est dédié à l'analyse de données, au Big Data et à l'intelligence artificielle. Il offre un environnement adapté aux activités de traitement et d'exploitation des données.

Espace Creativity

L'espace **Creativity** encourage l'idéation, la créativité et la collaboration interdisciplinaire.

Il complète les espaces techniques en favorisant la conception de nouveaux projets et solutions.

1.1.8 Écosystème, clubs et partenariats

Code212-UCA développe un écosystème réunissant étudiants, clubs universitaires, enseignants, experts et entreprises afin de renforcer les liens entre formation, innovation et monde professionnel.

Partenariats avec le monde professionnel

Le centre collabore avec plusieurs acteurs technologiques et industriels intervenant notamment dans les réseaux, la cybersécurité, le Cloud Computing et les systèmes d'information.

Ces partenariats peuvent contribuer au développement de formations, de certifications, d'événements, de projets ou encore d'opportunités de stage et d'insertion professionnelle.

1.1.9 Vision de Code212-UCA

La vision de Code212-UCA consiste à contribuer au développement d'une université plus connectée aux transformations technologiques et aux besoins du marché de l'emploi.

Le centre cherche à enrichir les cursus universitaires à travers des compétences complémentaires, des certifications et des expériences pratiques.

Cette démarche vise à favoriser l'employabilité, la polyvalence et l'adaptabilité des étudiants face à l'évolution rapide des métiers.

1.1.10 Code212-UCA comme cadre du Projet de Fin d'Année

Le Centre Code212-UCA constitue un environnement adapté à la réalisation de projets orientés vers les technologies émergentes et la transformation numérique.

Le Projet de Fin d'Année réalisé dans le cadre de ce stage porte sur la conception et la mise en œuvre d'une solution de gestion des accès privilégiés *Just-In-Time*, à l'intersection de la cybersécurité, de l'IAM et du Cloud Computing.

Il mobilise notamment des concepts liés à la gestion des identités, aux politiques d'autorisation, à l'authentification multifacteur, aux privilèges temporaires et à la supervision des événements de sécurité.

Sa réalisation au sein de Code212 s'inscrit ainsi dans la philosophie du centre, qui associe apprentissage, expérimentation et développement de solutions répondant à des problématiques technologiques actuelles. “‘

1.2 Cadre Institutionnel

1.2.1 École Nationale des Sciences Appliquées de Marrakech

L'École Nationale des Sciences Appliquées (ENSA) de Marrakech, fondée en 2000, a pour mission de former des ingénieurs hautement qualifiés à travers une pédagogie active favorisant l'ouverture professionnelle, internationale et culturelle. Affiliée à l'Université Cadi Ayyad, l'ENSA de Marrakech joue un rôle majeur dans la formation d'ingénieurs capables de répondre aux exigences du monde industriel et technologique.

Figure 1.2 – Logo de ENSA Marrakech

1.2.2 Filière GCDSTE

La filière Génie Cyberdéfense et Systèmes de Télécommunications Embar qués (GCD-STE) forme des ingénieurs dotés de solides compétences en cybersécurité et en systèmes de télécommunications. Cette formation intègre des méthodes avancées de protection des systèmes d'information, de gestion des risques numériques et de conception de solutions sécurisées. C'est dans ce cadre pédagogique que s'inscrit le présent projet de plateforme de gestion des accès privilégiés Just-In-Time. "'latex

1.3 Évolution de la Gestion des Identités et des Accès

1.3.1 Historique et tendances

La gestion des identités et des accès (*IAM*) a évolué avec la transformation des systèmes d'information et l'adoption du Cloud Computing. Les mécanismes traditionnels, basés sur des comptes et des politiques statiques, ont progressivement laissé place à des solutions plus avancées comme l'authentification multifacteur, le *Single Sign-On*, la fédération des identités et les contrôles d'accès basés sur les rôles ou les attributs.

Les approches actuelles privilégient des accès plus dynamiques, fondés sur le moindre privilège et la limitation temporelle des autorisations. Le *Just-In-Time Access* s'inscrit dans cette logique en accordant les privilèges uniquement lorsqu'ils sont nécessaires.

1.3.2 Risques liés aux privilèges permanents

Dans de nombreuses infrastructures, certains utilisateurs conservent des privilèges élevés même lorsqu'ils ne sont pas utilisés. Ces autorisations permanentes, appelées *Standing Privileges*, augmentent considérablement la surface d'attaque.

Les principaux risques associés sont :

— compromission d'un compte disposant de permissions élevées ;
— maintien d'autorisations après la disparition du besoin initial ;
— attribution de privilèges supérieurs aux besoins réels ;
— difficulté de révocation et risque d'erreurs administratives ;
— manque de traçabilité sur l'utilisation effective des privilèges ;
— augmentation de la surface d'attaque dans les environnements Cloud.

Ces risques sont particulièrement importants dans le Cloud, où une permission excessive peut permettre d'accéder rapidement à des ressources sensibles ou d'en modifier la configuration.

1.3.3 Émergence du Just-In-Time Access

Le *Just-In-Time Access* vise à remplacer les privilèges permanents par des autorisations temporaires. L'utilisateur ne possède aucun droit sensible par défaut, mais effectue une demande lorsque l'accès devient nécessaire.

La décision d'accès peut prendre en considération l'identité de l'utilisateur, son rôle, la ressource demandée, la justification, la durée souhaitée, le contexte de la demande et une authentification renforcée.

Cette approche est étroitement liée au principe de *Zero Standing Privilege*, dont l'objectif est de réduire au maximum l'existence de privilèges permanents et de limiter leur durée d'exposition.

1.3.4 Lien avec le projet

Le projet réalisé s'inscrit directement dans cette évolution. Il consiste à développer une plateforme capable d'automatiser l'attribution de privilèges temporaires à des ressources Cloud.

La solution combine plusieurs mécanismes complémentaires : authentification de l'utilisateur, validation d'une justification, décision d'autorisation basée sur des politiques, authentification multifacteur, génération d'identifiants AWS temporaires et supervision des événements de sécurité.

Les principaux composants utilisés sont **Keycloak**, **Jira**, **Open Policy Agent**, **AWS STS** et **Splunk**.

1.4 Problématique et Enjeux

1.4.1 Limites des approches traditionnelles

Les approches traditionnelles de gestion des privilèges reposent souvent sur des autorisations attribuées à l'avance et conservées jusqu'à leur révocation manuelle. Ce fonctionnement présente plusieurs limites :

— présence de privilèges actifs alors qu'ils ne sont pas nécessaires ;
— dépendance à une intervention administrative pour attribuer ou retirer les droits ;
— difficulté à vérifier automatiquement la justification d'une demande ;
— absence de limitation systématique de la durée des accès ;
— risque d'utilisation d'identifiants permanents ;
— difficulté à centraliser les décisions et les événements associés.

Ces limites rendent la gestion des privilèges plus complexe et augmentent les risques de sécurité, en particulier dans les infrastructures Cloud.

1.4.2 Défis liés à la gestion dynamique des accès

La mise en place d'un système Just-In-Time nécessite de résoudre plusieurs problématiques. Il faut tout d'abord garantir que le demandeur est correctement authentifié, mais aussi vérifier que sa demande correspond à un besoin légitime.

La décision d'autorisation doit également prendre en compte différents attributs tels que le rôle de l'utilisateur, le rôle demandé, la durée de l'accès et les informations associées à la justification.

Un autre défi concerne l'étape précédant l'octroi des privilèges. Une authentification supplémentaire est nécessaire afin de réduire le risque qu'une session déjà ouverte soit utilisée de manière frauduleuse.

Enfin, les identifiants temporaires générés doivent être distribués de manière sécurisée, sans être inutilement exposés dans l'interface utilisateur ou dans les journaux.

1.4.3 Enjeux du projet

Les principaux enjeux du projet sont les suivants :

— réduire l'utilisation de privilèges permanents ;
— attribuer uniquement les droits réellement nécessaires ;
— vérifier la légitimité de chaque demande ;
— centraliser les règles d'autorisation ;
— renforcer l'authentification avant l'accès privilégié ;
— limiter automatiquement la durée des autorisations ;
— protéger la distribution des identifiants temporaires ;
— assurer la traçabilité et la supervision du workflow ;
— automatiser autant que possible le processus d'accès.

1.5 Approche Proposée

1.5.1 Architecture fonctionnelle

Afin de répondre aux enjeux identifiés, la solution repose sur une architecture modulaire dans laquelle chaque composant assure une fonction précise.

Le processus débute par l'authentification de l'utilisateur à travers **Keycloak**. Celui-ci soumet ensuite une demande précisant le rôle temporaire souhaité, la durée d'accès et un ticket **Jira** servant de justification.

Les informations de la demande sont transmises à **Open Policy Agent (OPA)**, qui applique des politiques écrites en Rego selon une approche *Policy as Code*. La décision repose notamment sur le rôle de l'utilisateur, les caractéristiques du ticket et la durée

demandée.

Lorsque la demande est autorisée, une authentification multifacteur basée sur **TOTP** est imposée. Cette étape de *Step-Up Authentication* renforce la sécurité avant l'octroi des privilèges.

Le backend utilise ensuite **AWS Security Token Service (STS)** afin d'assumer un rôle IAM prédéfini et de générer des identifiants temporaires dont la durée de validité est limitée.

Afin de réduire leur exposition, ces identifiants ne sont pas directement affichés dans l'interface Web. Ils sont récupérés par un **agent local** à travers une communication sécurisée en HTTPS/TLS puis injectés dans une session PowerShell dédiée, permettant à l'utilisateur d'exécuter les commandes AWS CLI correspondant aux permissions temporaires obtenues.

Enfin, **Splunk** centralise les principaux événements de sécurité générés par la plateforme afin d'assurer la traçabilité, l'audit et la supervision du processus.

1.6 Périmètre et Contraintes

1.6.1 Périmètre fonctionnel

Le périmètre fonctionnel du projet couvre les éléments suivants :

— authentification des utilisateurs via Keycloak ;
— soumission d'une demande d'accès temporaire ;
— vérification d'un ticket Jira servant de justification ;
— prise de décision à travers OPA et des politiques Rego ;
— authentification multifacteur TOTP ;
— génération d'identifiants temporaires avec AWS STS ;
— attribution de permissions à travers des rôles IAM ;
— récupération sécurisée des identifiants via un agent local ;
— utilisation des accès temporaires depuis AWS CLI ;
— journalisation et supervision des événements avec Splunk.

1.6.2 Éléments exclus du périmètre

Le projet constitue une implémentation fonctionnelle destinée à valider le principe d'une architecture Just-In-Time. Certains éléments d'une solution industrielle complète ne sont donc pas couverts, notamment :

— le déploiement multi-cloud ;
— la haute disponibilité des composants ;
— l'intégration complète d'un Active Directory d'entreprise ;

— la gestion de plusieurs organisations et comptes Cloud complexes ;
— l'intégration d'une solution PAM industrielle complète ;
— le déploiement à grande échelle sur de nombreux postes utilisateurs.

1.6.3 Contraintes identifiées

La réalisation du projet a nécessité de prendre en compte plusieurs contraintes.

Contraintes techniques

L'intégration de plusieurs composants reposant sur des technologies différentes impose la gestion de plusieurs API, protocoles et mécanismes d'authentification. L'exécution simultanée de services tels que Keycloak, OPA et Splunk dans un environnement Docker local représente également une contrainte en termes de ressources matérielles.

La configuration d'AWS nécessite par ailleurs une définition précise des politiques IAM, des rôles et de leurs relations de confiance.

Contraintes de sécurité

La manipulation d'identifiants AWS temporaires nécessite de limiter leur exposition. Les secrets ne doivent notamment pas être enregistrés dans les logs ni être transmis sur un canal non sécurisé.

La communication entre le backend et l'agent local doit donc être protégée par HTTPS/TLS, tandis que les événements envoyés à Splunk doivent contenir uniquement les informations nécessaires à l'audit.

Contraintes temporelles

Le projet étant réalisé dans le cadre d'un stage PFA de durée limitée, la priorité a été accordée à la mise en œuvre et à la validation des fonctionnalités principales du workflow : authentification, autorisation, MFA, génération des accès temporaires, distribution sécurisée et supervision.

Conclusion

Ce chapitre a permis de présenter le contexte institutionnel et technologique du projet ainsi que l'évolution des mécanismes de gestion des identités et des accès. L'analyse des privilèges permanents a mis en évidence les risques liés au maintien d'autorisations sensibles au-delà du besoin réel.

L'approche Just-In-Time retenue vise à répondre à cette problématique en combinant authentification, justification des demandes, politiques d'autorisation, MFA, privilèges

Cloud temporaires et supervision. Le périmètre et les principales contraintes du projet ayant été définis, le chapitre suivant présentera plus en détail les concepts, technologies et choix d'architecture ayant permis la réalisation de la solution. "'

Chapitre 2

Cadre Théorique et État de l'Art

Introduction

La conception d'un système de gestion des accès temporaires mobilise un ensemble de concepts issus de plusieurs sous-domaines de la sécurité informatique : la gestion des identités et des accès (IAM), les protocoles d'authentification et d'autorisation normalisés, les modèles théoriques de contrôle d'accès, l'authentification renforcée, ainsi que les mécanismes propres aux environnements cloud pour la délégation d'identité. Ce chapitre a pour objectif de poser les bases théoriques nécessaires à la compréhension des choix d'architecture présentés dans les chapitres suivants, et de situer le projet par rapport à l'état de l'art et aux solutions existantes sur le marché.

Le chapitre est organisé en six sections. La première introduit les notions fondamentales de la gestion des identités et des accès, en particulier le principe de *Zero Standing Privilege* qui constitue le fondement conceptuel du projet. La deuxième détaille les protocoles d'authentification et d'autorisation normalisés (OAuth 2.0, OpenID Connect) sur lesquels repose l'authentification des utilisateurs. La troisième section présente les modèles théoriques de contrôle d'accès, du RBAC classique à l'ABAC, ainsi que le paradigme du *policy-as-code*. La quatrième section aborde les mécanismes d'authentification renforcée, et notamment l'algorithme TOTP. La cinquième section couvre les concepts propres à la gestion des identités dans le cloud, en particulier la délégation temporaire de privilèges. La sixième section traite des principes d'observabilité et d'audit en sécurité. Le chapitre se conclut par un positionnement du projet par rapport aux solutions commerciales existantes.

2.1 Gestion des identités et des accès (IAM)

2.1.1 Identity and Access Management : définitions générales

La gestion des identités et des accès (*Identity and Access Management*, IAM) désigne l'ensemble des politiques, processus et technologies permettant de garantir que les bonnes entités (utilisateurs, services, machines) disposent des bons accès, aux bonnes ressources, au bon moment, et pour les bonnes raisons. Un système IAM répond typiquement à quatre questions fondamentales : qui est l'entité qui se présente, quelles ressources

peut-elle solliciter, dans quelles conditions cet accès est-il légitime, et comment cet accès peut-il être tracé et révoqué.

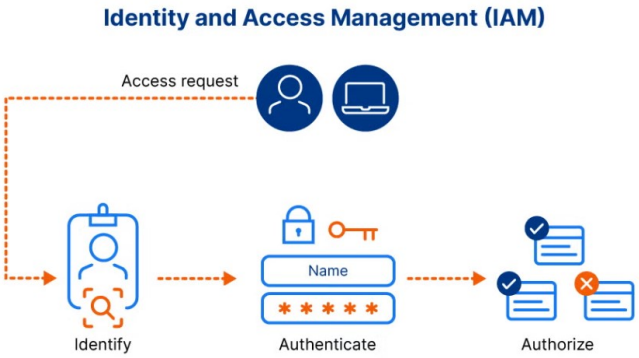


Figure 2.1 – Étapes principales d'un processus IAM : identification, authentification et autorisation

Un acteur central de tout système IAM est le *Identity Provider* (IdP), une entité de confiance responsable de l'authentification des utilisateurs et de la délivrance de preuves d'identité vérifiables (jetons), indépendamment des applications qui consomment ensuite ces preuves. Cette centralisation permet la mise en œuvre du *Single Sign-On* (SSO), où un utilisateur s'authentifie une seule fois auprès de l'IdP pour accéder à plusieurs applications sans ressaisir ses identifiants.

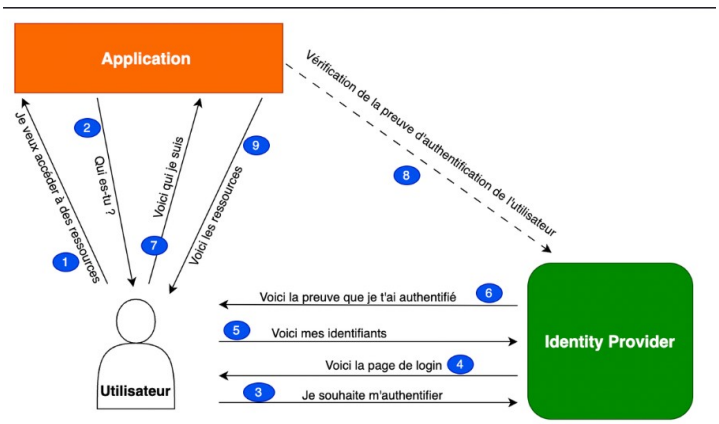


Figure 2.2 – Flux d'authentification entre l'utilisateur , l'application et le fournisseur d'identité

2.1.2 Authentification versus autorisation

Il est essentiel de distinguer deux notions fréquemment confondues dans le langage courant mais rigoureusement séparées sur le plan conceptuel :

— **L'authentification** (*Authentication*, AuthN) répond à la question « qui êtes-vous ? ». Elle consiste à vérifier qu'une entité est bien celle qu'elle prétend être, généralement par la présentation d'un ou plusieurs facteurs de preuve.

— **L'autorisation** (*Authorization*, AuthZ) répond à la question « avez-vous le droit de faire ceci ? ». Elle intervient *après* l'authentification et détermine, pour une identité déjà vérifiée, quelles actions ou ressources lui sont accessibles.

Cette séparation constitue un principe architectural fondamental : un système bien conçu ne doit jamais mélanger la logique de vérification d'identité avec la logique de décision d'accès, ces deux responsabilités devant pouvoir évoluer, être auditées et être remplacées indépendamment l'une de l'autre.



Figure 2.3 – Authentification VS autorisation

2.1.3 Le problème des privilèges permanents

Dans la majorité des systèmes d'information traditionnels, les droits d'accès privilégiés sont attribués de façon *permanente* : un administrateur système dispose en continu de droits d'administration, qu'il les utilise une fois par mois ou plusieurs fois par jour. On parle alors de *standing privileges* (privilèges permanents ou « dormants »).

Cette pratique présente un risque de sécurité majeur, largement documenté dans la littérature et les référentiels de sécurité (NIST SP 800-207, recommandations CISA) : si le compte porteur de ces privilèges est compromis — par hameçonnage, vol de session ou fuite d'identifiants — l'attaquant hérite immédiatement de l'intégralité des droits associés, sans limite de temps et sans qu'aucune action supplémentaire ne soit requise. Les analyses post-incident de nombreuses fuites de données majeures identifient de façon récurrente la compromission d'un compte à privilèges permanents comme vecteur d'escalade et de mouvement latéral.

2.1.4 Le principe de Zero Standing Privilege

Le principe de *Zero Standing Privilege* (ZSP) propose une inversion de ce modèle : **aucun** privilège élevé ne doit exister par défaut. À tout instant, un audit du système devrait révéler qu'aucun utilisateur ne dispose de droits d'administration « au repos » ;

ces droits ne doivent apparaître que ponctuellement, pour la durée strictement nécessaire à l'accomplissement d'une tâche légitime, puis disparaître automatiquement.

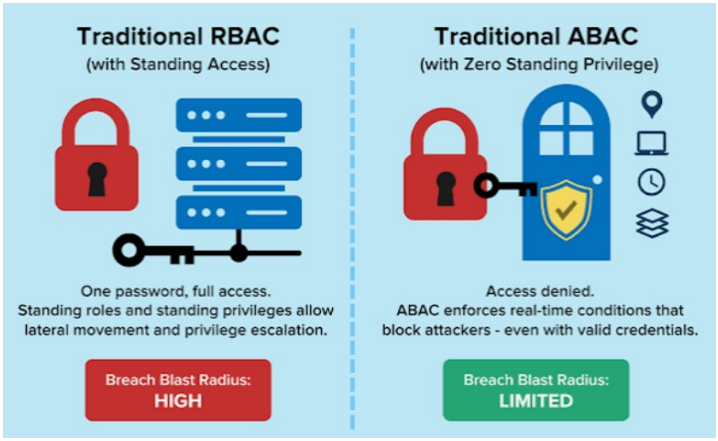


Figure 2.4 – Comparaison entre le modèle de privilèges permanents (standing privileges) et le modèle Zero Standing Privilege

2.1.5 Just-In-Time Access

Le *Just-In-Time Access* (JIT Access) constitue la mise en œuvre opérationnelle du principe ZSP : l'octroi d'un droit se produit exactement au moment où le besoin se manifeste, et non en amont de façon anticipée. Un système JIT typique repose sur quatre étapes : la demande explicite d'un accès par l'utilisateur, la vérification d'une justification légitime, une décision d'autorisation (automatisée ou soumise à validation humaine), et enfin la révocation automatique à l'expiration d'une durée prédéfinie.

Le JIT Access se distingue ainsi du contrôle d'accès statique traditionnel par sa dimension *temporelle* : la question n'est plus seulement « cet utilisateur a-t-il le droit d'accéder à cette ressource ? », mais « cet utilisateur a-t-il, *à cet instant précis et pour cette durée précise*, une raison légitime d'y accéder ? ».



Figure 2.5 – Principe du Just-In-Time Access

2.1.6 Le principe du moindre privilège

Le *principe du moindre privilège* (*Least Privilege Principle*), formulé dès les premiers travaux sur la sécurité des systèmes multi-utilisateurs, énonce qu'une entité ne doit disposer que des permissions strictement nécessaires à l'accomplissement d'une tâche

donnée, ni plus, ni moins. Ce principe se décline sur plusieurs dimensions complémentaires au JIT Access :

— **Dimension temporelle** : le droit n'existe que pendant la durée nécessaire (apport du JIT Access).
— **Dimension fonctionnelle** : le droit ne couvre que l'action strictement requise (par exemple une permission d'écriture seule, sans droit de suppression).
— **Dimension du périmètre** : le droit ne s'applique qu'à la ressource précise concernée, et non à l'ensemble d'un système.

Ces trois dimensions se retrouvent conjointement dans la conception d'un système JIT Access abouti, où chaque rôle temporaire accordé est à la fois borné dans le temps, restreint à une action précise, et limité à une ressource identifiée.

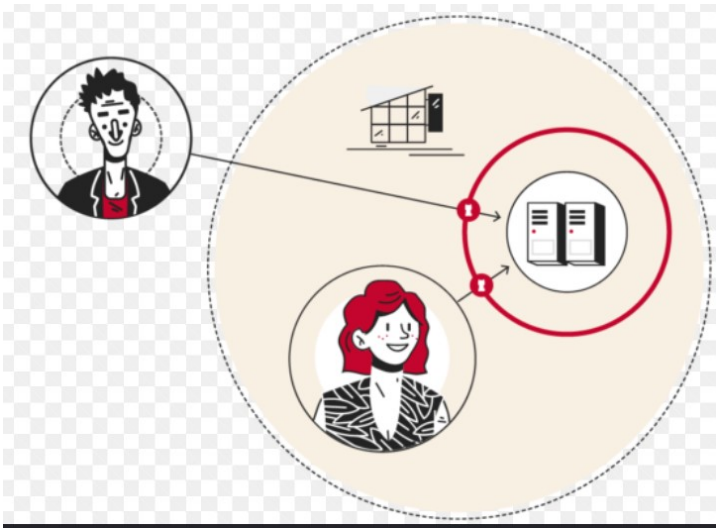


Figure 2.6 – Le principe du moindre privilège

2.2 Protocoles d'authentification et d'autorisation

2.2.1 OAuth 2.0

OAuth 2.0 (RFC 6749) est un cadre de délégation d'autorisation permettant à une application tierce d'obtenir un accès limité à une ressource protégée, au nom d'un utilisateur, *sans que ce dernier ne partage jamais son mot de passe* avec l'application demandeuse. Le protocole définit quatre rôles distincts :

— le **propriétaire de la ressource** (*resource owner*), généralement l'utilisateur final ;
— le **client**, l'application qui souhaite accéder à la ressource ;
— le **serveur d'autorisation** (*authorization server*), qui authentifie l'utilisateur et délivre des jetons ;

— le **serveur de ressources** (*resource server*), qui héberge la ressource protégée et valide les jetons présentés.

OAuth 2.0 définit plusieurs *flows* (parcours d'obtention de jeton) adaptés à différents contextes applicatifs, dont deux sont particulièrement pertinents dans le cadre de ce projet et détaillés ci-après.

2.2.2 OpenID Connect

OAuth 2.0, dans sa conception originelle, est un protocole d'*autorisation* et ne définit pas nativement de mécanisme normalisé pour transmettre l'identité de l'utilisateur authentifié. **OpenID Connect** (OIDC) est une surcouche construite au-dessus d'OAuth 2.0 qui comble cette lacune en introduisant la notion de *jeton d'identité* (*ID Token*), un jeton signé au format JWT (voir §2.2.4) contenant des informations vérifiables sur l'utilisateur authentifié (identifiant unique, adresse électronique, date d'expiration, émetteur).

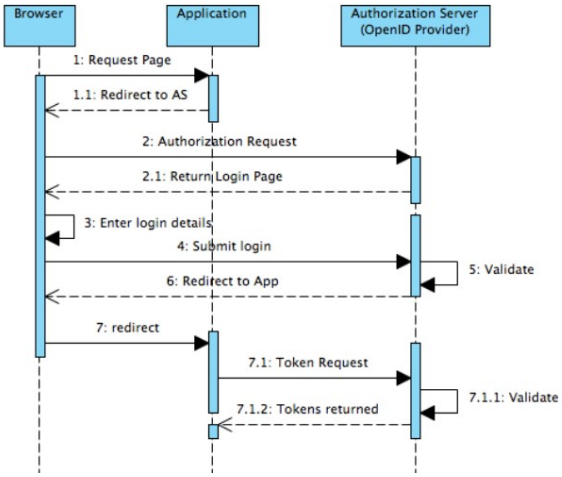


Figure 2.7 – Flux du protocole OpenID Connect

2.2.3 Le flow Authorization Code avec PKCE

Le *Authorization Code Flow* est le parcours OAuth2/OIDC recommandé pour les applications disposant d'un composant serveur ou exécutées dans un navigateur. Son déroulement se résume comme suit :

1. L'utilisateur est redirigé depuis l'application cliente vers le serveur d'autorisation.
2. L'utilisateur s'authentifie directement auprès du serveur d'autorisation (jamais auprès de l'application cliente).
3. Le serveur d'autorisation redirige l'utilisateur vers l'application cliente en y joignant un *code d'autorisation* temporaire à usage unique.
4. L'application cliente échange ce code, en arrière-plan, contre un jeton d'accès et un jeton d'identité.

Pour les applications dites *publiques* — c'est-à-dire ne pouvant garder aucun secret confidentiel, typiquement une application exécutée entièrement côté navigateur — une extension nommée **PKCE** (*Proof Key for Code Exchange*, RFC 7636) est requise. PKCE introduit un secret généré dynamiquement à chaque tentative de connexion (le *code verifier*) et sa version dérivée transmise en clair (le *code challenge*), afin de garantir que seule l'application ayant initié la demande de connexion pourra effectivement échanger le code d'autorisation contre un jeton, neutralisant ainsi le risque d'interception de ce code par une application tierce malveillante installée sur le même appareil.

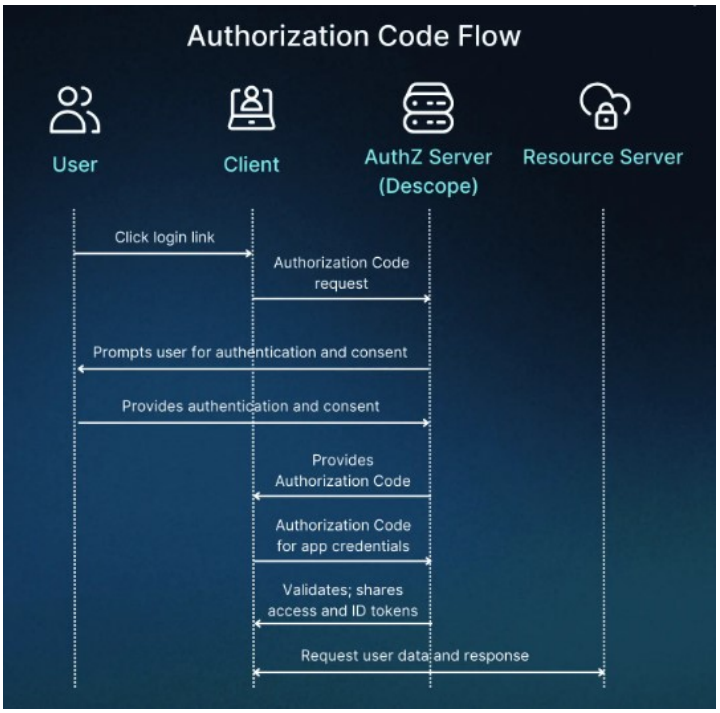


Figure 2.8 – Séquence du flow Authorization Code avec PKCE

2.2.4 JSON Web Token et JWKS

Un **JSON Web Token** (JWT, RFC 7519) est un format de jeton compact et auto-porteur, structuré en trois parties encodées séparément et concaténées par des points : un en-tête (*header*) précisant l'algorithme de signature utilisé, une charge utile (*payload*) contenant les informations (*claims*) relatives au sujet du jeton, et une signature cryptographique garantissant l'intégrité des deux premières parties.

$$\text{JWT} = \text{Base64Url}(\text{header}) \,.\, \text{Base64Url}(\text{payload}) \,.\, \text{Base64Url}(\text{signature}) \tag{2.1}$$

La signature d'un JWT peut être vérifiée par tout tiers possédant la clé publique correspondante, *sans avoir besoin de recontacter le serveur d'autorisation émetteur*, ce qui confère au JWT sa propriété d'auto-suffisance et permet une validation locale

performante côté serveur de ressources. Ces clés publiques sont exposées de façon normalisée via un point de terminaison **JWKS** (*JSON Web Key Set*), une collection de clés publiques identifiées chacune par un identifiant (*key ID*, `kid`) permettant au vérificateur de sélectionner la clé correspondant à celle utilisée pour signer un jeton donné.

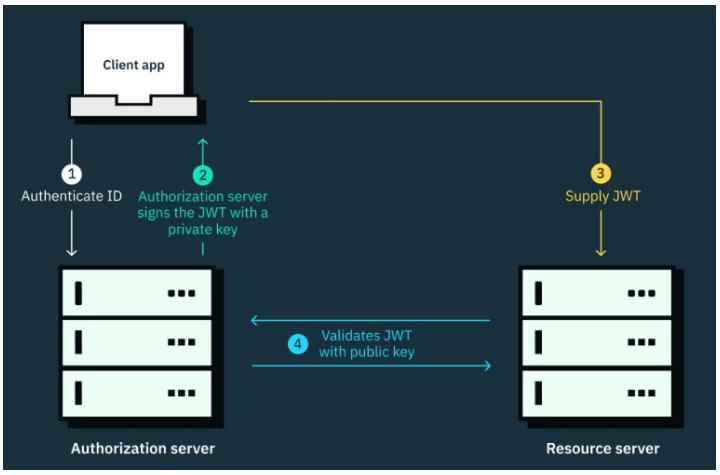


Figure 2.9 – Vérification de la signature du JSON Web Token via JWKS

2.3 Modèles théoriques de contrôle d'accès

2.3.1 Role-Based Access Control (RBAC)

Le modèle **RBAC** (*Role-Based Access Control*), formalisé notamment par Ferraiolo et Kuhn puis normalisé par le NIST, organise les décisions d'accès autour de la notion de *rôle* : plutôt que d'attribuer des permissions individuellement à chaque utilisateur, on définit des rôles porteurs d'un ensemble de permissions, et l'on affecte chaque utilisateur à un ou plusieurs rôles. Ce modèle simplifie considérablement l'administration des droits dans les organisations de grande taille, en découplant la gestion des permissions (rarement modifiée) de la gestion des affectations utilisateur-rôle (fréquemment modifiée).

Le RBAC présente cependant une limite structurelle : la décision d'accès repose exclusivement sur l'appartenance à un rôle, sans possibilité native de prendre en compte des éléments *contextuels* (l'heure de la demande, l'état d'une ressource externe, la criticité d'une situation particulière).

2.3.2 Attribute-Based Access Control (ABAC)

Le modèle **ABAC** (*Attribute-Based Access Control*) généralise le RBAC en fondant la décision d'accès sur l'évaluation combinée de plusieurs *attributs*, pouvant relever du sujet (rôle, département, ancienneté), de la ressource (sensibilité, propriétaire), de l'action demandée, et de l'environnement (heure, localisation, niveau de risque contextuel). Une

politique ABAC prend typiquement la forme d'une règle logique combinant plusieurs conditions, qui doivent *toutes* être satisfaites — ou satisfaites selon une combinaison logique définie — pour que l'accès soit autorisé.

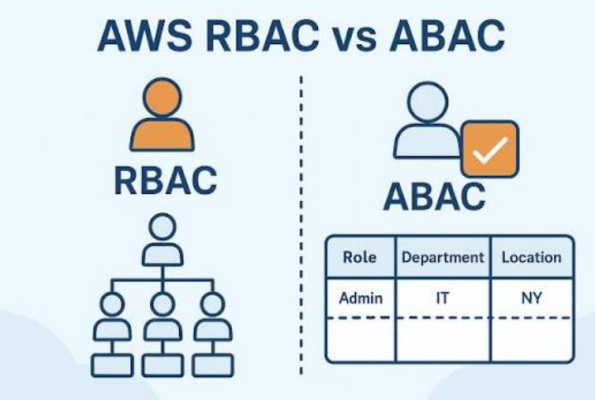


Figure 2.10 – Comparaison structurelle entre les modèles RBAC et ABAC

L'ABAC offre une expressivité nettement supérieure au RBAC pour modéliser des politiques d'accès conditionnelles et contextuelles, au prix d'une complexité de conception et d'audit accrue : la multiplication des attributs combinables rend plus difficile la prévision exhaustive de l'ensemble des cas couverts par une politique donnée, d'où l'importance des outils de *policy-as-code* permettant de tester ces règles de façon systématique.

2.3.3 Policy-as-Code et Open Policy Agent

Le paradigme du **policy-as-code** consiste à exprimer les règles de décision d'accès sous forme de code source, versionnable, testable indépendamment de l'application, et découplé du code métier de celle-ci. Cette approche s'oppose à la pratique consistant à coder ces règles « en dur » au sein même de la logique applicative, pratique qui rend les règles difficiles à auditer, à faire évoluer sans redéploiement, et à tester isolément.

Open Policy Agent (OPA) est un moteur de politique générique et agnostique du domaine d'application, permettant d'évaluer des décisions d'autorisation à partir de règles écrites dans un langage déclaratif dédié nommé **Rego**.

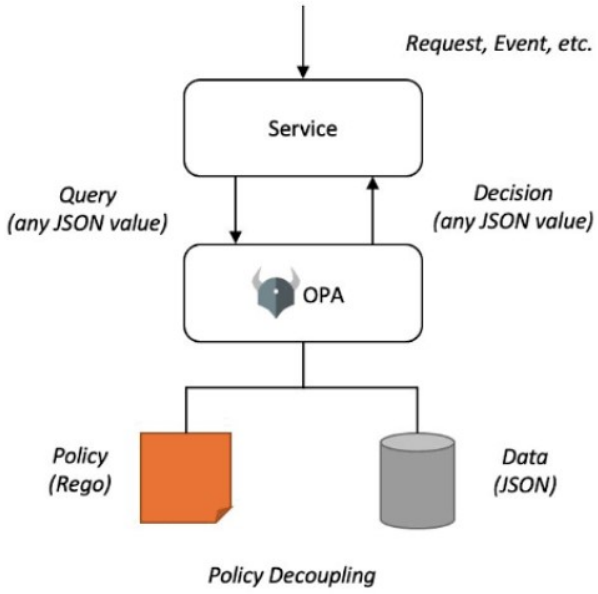


Figure 2.11 – Principe de fonctionnement d'Open Policy Agent dans une architecture policy-as-code

Cette architecture découplée présente plusieurs avantages significatifs : la logique de décision peut être testée unitairement de façon indépendante de l'application ; elle peut être versionnée et auditée au même titre que le code source ; elle peut évoluer sans nécessiter de redéploiement de l'application elle-même dans le cas d'un service OPA déployé séparément ; enfin, elle favorise la réutilisation de règles communes à travers plusieurs applications distinctes d'une même organisation.

2.4 Concepts cloud et gestion des identités machine

2.4.1 Identités permanentes versus identités fédérées

Les fournisseurs cloud distinguent généralement deux grandes catégories d'identités programmatiques. Les **identités permanentes** (*long-term credentials*) consistent en une paire de clés statiques, valides jusqu'à révocation manuelle explicite ; leur simplicité d'usage s'accompagne d'un risque structurel similaire à celui des privilèges permanents évoqué en §2.1 : une fuite de ces clés octroie un accès valide sans limite de temps. Les **identités fédérées ou temporaires** (*federated / temporary credentials*), à l'inverse, sont émises dynamiquement pour une durée bornée et deviennent automatiquement invalides à l'expiration de cette période, sans qu'aucune action de révocation explicite ne soit nécessaire.

2.4.2 Le mécanisme AssumeRole

Les fournisseurs cloud proposent généralement un service dédié à l'émission d'identités temporaires par délégation, fonctionnant selon le principe de l'*endossement de rôle* (*AssumeRole*) : une identité initiale, disposant d'un droit limité et explicite d'endosser un rôle cible, échange ce droit contre un jeu de credentials temporaires porteurs des permissions associées à ce rôle, sans jamais recevoir les identifiants permanents originaux de ce rôle.

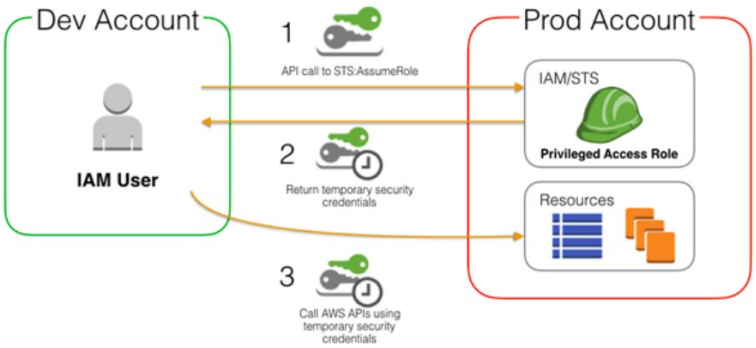


Figure 2.12 – Principe général du mécanisme d'endossement temporaire de rôle (AssumeRole)

Ce mécanisme repose sur une relation de confiance *bidirectionnelle*, formalisée par deux documents distincts et complémentaires :

— une **politique de confiance** (*Trust Policy*), attachée au rôle cible, qui définit *quelles identités sont autorisées à l'endosser* ;
— une **politique de permissions** (*Permission Policy*), également attachée au rôle, qui définit *quelles actions ce rôle autorise une fois endossé.*

L'endossement n'aboutit que si les deux conditions sont simultanément satisfaites : l'identité source doit explicitement demander à endosser ce rôle précis, et le rôle doit explicitement accepter d'être endossé par cette identité précise. Cette double vérification constitue une application directe du principe de séparation des devoirs évoqué en §2.3.

2.4.3 Expiration cryptographique des credentials

Une propriété fondamentale distingue les credentials temporaires émis par un service de jetons de sécurité d'une simple règle applicative de péremption : leur expiration est **garantie par le fournisseur cloud lui-même**, indépendamment de tout comportement de l'application émettrice. Un jeton temporaire encode cryptographiquement sa propre date d'expiration ; toute tentative d'utilisation de ce jeton après cette date est rejetée par le fournisseur cloud au niveau de son infrastructure d'authentification, sans qu'aucune action de révocation explicite ni aucune disponibilité de l'application ayant émis le jeton

ne soit requise. Cette garantie constitue le socle technique sur lequel repose la fiabilité du principe de Zero Standing Privilege appliqué aux environnements cloud : même en cas de défaillance totale du système d'octroi d'accès après l'émission d'un jeton, la fenêtre d'exposition demeure strictement bornée par la durée initialement définie.

2.5 Observabilité et sécurité opérationnelle

2.5.1 Security Information and Event Management (SIEM)

Un **SIEM** (*Security Information and Event Management*) désigne une catégorie de plateformes dédiées à la collecte centralisée, la corrélation et l'analyse des événements de sécurité générés par les différents composants d'un système d'information. Un SIEM permet notamment la détection d'anomalies comportementales par agrégation d'événements dispersés dans le temps ou entre plusieurs sources — une capacité difficilement réalisable par l'examen isolé des journaux d'un unique composant — ainsi que le déclenchement d'alertes automatisées lorsque des motifs prédéfinis, potentiellement révélateurs d'une tentative de contournement ou d'abus, sont détectés.

2.5.2 Le principe de non-interférence de la couche d'observabilité

Une considération architecturale importante concerne le positionnement de la couche d'observabilité par rapport à la logique de décision métier. Un principe de conception largement admis établit que l'indisponibilité d'un système d'observabilité ne doit **jamais** empêcher le déroulement normal d'une opération métier légitime : la fonction d'un SIEM est d'observer et d'alerter *a posteriori* ou en parallèle, non de conditionner *a priori* l'issue d'une décision d'autorisation, rôle qui incombe exclusivement au moteur de décision dédié (voir §2.3).

2.5.3 Piste d'audit

La *piste d'audit* (*audit trail*) désigne l'enregistrement chronologique, exhaustif et non modifiable a posteriori des événements et décisions significatifs d'un système. Au-delà de sa valeur pour l'investigation post-incident, la piste d'audit constitue une exigence explicite de nombreux référentiels de conformité en sécurité de l'information (normes ISO/CEI 27001, référentiel SOC 2), qui imposent notamment la capacité à démontrer, pour tout accès privilégié accordé, l'existence d'une justification métier documentée et tracée.

2.6 Positionnement par rapport à l'état de l'art

Le marché de la gestion des accès à privilèges (*Privileged Access Management*, PAM) propose de nombreuses solutions commerciales matures intégrant tout ou partie des concepts présentés dans ce chapitre, parmi lesquelles on peut citer CyberArk, HashiCorp Boundary, ou encore les fonctionnalités natives de gestion des identités privilégiées proposées par les grands fournisseurs cloud, telles qu'Azure Privileged Identity Management (PIM) ou AWS IAM Identity Center.

Ces solutions constituent des produits intégrés, généralement proposés sous forme de licence commerciale, prenant en charge l'ensemble du cycle de vie d'une demande d'accès temporaire au sein d'une interface unifiée. Le présent projet se distingue de ces offres par une approche de *composition* plutôt que d'*intégration* : chaque brique fonctionnelle — authentification, justification métier, décision d'autorisation, authentification renforcée, émission de credentials — est assurée par un composant spécialisé et interchangeable, orchestré par une couche applicative propre plutôt que consommé comme une boîte noire. Cette approche présente un intérêt pédagogique et architectural particulier : elle impose une compréhension fine de chacun des protocoles et mécanismes sous-jacents, et permet une démonstration explicite de la façon dont ces briques, individuellement standards et documentées, peuvent être composées pour aboutir à un système cohérent appliquant le principe de Zero Standing Privilege, sans dépendre d'une solution commerciale unique et fermée.

Conclusion

Ce chapitre a présenté les fondements théoriques mobilisés par le projet, en couvrant successivement les principes de gestion des identités et des accès — au premier rang desquels le Zero Standing Privilege et le Just-In-Time Access qui constituent le cœur conceptuel de ce travail —, les protocoles normalisés d'authentification et d'autorisation sur lesquels repose l'interopérabilité du système, les modèles théoriques de contrôle d'accès justifiant le recours à une approche par politique-as-code, ainsi que les concepts propres à la délégation d'identité en environnement cloud. Le chapitre suivant englobera les technologies utilisées.

Chapitre 3

Technologies Adoptées

Introduction

Ce chapitre présente les principales technologies utilisées pour la réalisation de la plateforme de gestion des accès privilégiés Just-In-Time. Elles couvrent les différentes fonctions du système : gestion des identités, validation des demandes, prise de décision, authentification renforcée, génération des accès AWS temporaires, distribution sécurisée des identifiants, journalisation et conteneurisation. Le rôle de chaque technologie ainsi que les raisons principales de son choix sont présentés dans les sections suivantes.

3.1 Keycloak — Gestion des identités et des accès

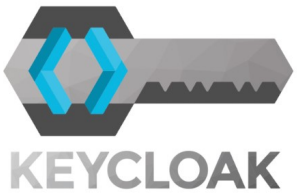


Figure 3.1 – Logo de Keycloak

Keycloak est une solution open source de gestion des identités et des accès. Elle prend notamment en charge les protocoles standards **OpenID Connect**, **OAuth 2.0** et **SAML 2.0**.

Dans ce projet, Keycloak joue le rôle de fournisseur d'identité (*Identity Provider*). Il assure l'authentification des utilisateurs et permet de gérer leurs comptes ainsi que leurs rôles. Après authentification, l'application peut exploiter les informations contenues dans le jeton délivré par Keycloak pour identifier l'utilisateur et transmettre les attributs nécessaires au processus d'autorisation.

L'interface Web utilise le protocole OpenID Connect avec le flux *Authorization Code* et PKCE. Cette approche est adaptée aux applications clientes qui ne peuvent pas protéger durablement un secret.

Le choix de Keycloak repose principalement sur son caractère open source, sa prise en charge des standards d'identité modernes, sa souplesse de configuration et sa possibilité

d'être déployé localement dans un environnement Docker.

3.2 Open Policy Agent et Rego — Moteur de décision



Figure 3.2 – Logo de Open Policy Agent

Open Policy Agent (OPA) est un moteur de politiques open source permettant d'externaliser les règles d'autorisation du code applicatif. Dans le projet, le backend interroge OPA à travers son interface HTTP afin d'obtenir une décision concernant une demande d'accès.

Les politiques sont écrites en **Rego**, un langage déclaratif permettant d'exprimer les conditions à respecter avant d'autoriser une demande. Les règles prennent notamment en compte des informations relatives à l'utilisateur, au rôle demandé, au ticket Jira et à la durée d'accès sollicitée.

Cette approche met en œuvre le principe de *Policy as Code* : les règles de sécurité sont définies séparément du backend, ce qui facilite leur modification, leur lecture et leur test.

OPA a été retenu pour sa capacité à centraliser la logique d'autorisation et à mettre en œuvre un contrôle d'accès dynamique basé sur plusieurs attributs.

3.3 Jira — Système de ticketing



Figure 3.3 – Logo de Jira

Jira, développé par Atlassian, est une plateforme de gestion de projets, de tickets et de suivi des tâches.

Dans la solution développée, Jira constitue la source de justification de la demande d'accès. L'utilisateur doit fournir la référence d'un ticket lors de sa demande. Le backend interroge ensuite l'API REST de Jira afin de récupérer les informations utiles du ticket et de les intégrer dans la décision d'autorisation.

L'utilisation d'un système de ticketing permet ainsi d'associer l'élévation temporaire de privilèges à un besoin identifiable et traçable, au lieu d'accorder un accès sans justification préalable.

3.4 Python et FastAPI — Backend applicatif



Figure 3.4 – Logos de Python et FastAPI

Le backend de la plateforme est développé en **Python** à l'aide du framework **FastAPI**. Il assure l'orchestration du workflow Just-In-Time et constitue le point de communication entre les différents composants de l'architecture.

Il prend notamment en charge la réception des demandes d'accès, la communication avec Jira et OPA, la vérification de l'authentification renforcée ainsi que l'interaction avec AWS STS.

FastAPI a été retenu pour sa simplicité de développement d'API REST, sa prise en charge de la validation des données avec **Pydantic**, son support des traitements asynchrones et sa génération automatique d'une documentation d'API interactive.

La bibliothèque **httpx** est utilisée pour effectuer les communications HTTP avec certains services externes, tandis que **Uvicorn** assure l'exécution de l'application FastAPI via le standard ASGI.

3.5 AWS IAM, STS et boto3 — Gestion des accès temporaires



Figure 3.5 – Logo de Amazon Web Services

Amazon Web Services (AWS) constitue l'environnement Cloud cible sur lequel les privilèges temporaires sont appliqués. Le projet exploite principalement les services **IAM** et **STS**.

— **AWS IAM** (*Identity and Access Management*) permet de définir les politiques de permissions, les rôles IAM ainsi que l'identité applicative utilisée par le backend.
— **AWS STS** (*Security Token Service*) permet de générer des identifiants temporaires en utilisant l'opération `AssumeRole`.

L'application backend utilise une identité IAM disposant uniquement de l'autorisation nécessaire pour assumer des rôles prédéfinis. Les permissions réellement accordées à l'utilisateur final sont définies par les politiques attachées au rôle assumé.

Dans le cadre du projet, des rôles associés à différentes permissions sur **Amazon S3** permettent d'illustrer la séparation des privilèges, par exemple entre des opérations de lecture et d'écriture.

Après l'appel à `AssumeRole`, AWS STS retourne trois éléments principaux :

— `AccessKeyId` ;
— `SecretAccessKey` ;
— `SessionToken`.

Ces identifiants possèdent une durée de validité limitée. Une fois expirés, ils ne permettent plus d'effectuer d'opérations AWS.

L'intégration avec AWS est réalisée à l'aide de **boto3**, le SDK AWS officiel pour Python. Il permet notamment au backend d'appeler STS et d'exécuter programmatiquement l'opération `AssumeRole`.

Le choix d'AWS repose principalement sur la maturité de ses mécanismes IAM, la disponibilité de STS pour la génération de sessions temporaires ainsi que sur la documentation détaillée associée à la gestion des rôles et des politiques.

3.6 TOTP et pyotp — Authentification renforcée



Figure 3.6 – Microsoft Authenticator

Avant l'émission des identifiants AWS temporaires, une étape d'authentification renforcée est imposée à l'utilisateur.

Cette vérification repose sur le mécanisme **TOTP** (*Time-based One-Time Password*), défini par la RFC 6238. Un code temporaire est généré à partir d'un secret partagé et du temps courant, puis renouvelé périodiquement.

La bibliothèque Python **pyotp** est utilisée côté backend pour gérer la génération et la vérification des codes TOTP. Côté utilisateur, une application compatible telle que **Microsoft Authenticator** permet de générer le code demandé.

L'utilisation de cette deuxième vérification juste avant l'attribution des privilèges constitue une forme de *step-up authentication*. Elle réduit le risque qu'une session applicative compromise suffise à obtenir directement un accès Cloud privilégié.

3.7 Splunk — Journalisation et supervision



Figure 3.7 – Logo de Splunk

Splunk est une plateforme permettant de centraliser, rechercher et analyser des événements et des journaux provenant de différentes sources. Elle peut notamment être utilisée dans des contextes de supervision et de SIEM (*Security Information and Event Management*).

Dans ce projet, Splunk est utilisé comme composant de **journalisation et de supervision**. Les événements significatifs du workflow JIT peuvent y être centralisés, notamment :

— les demandes d'accès ;
— les refus issus des politiques d'autorisation ;
— les échecs de l'authentification renforcée ;
— les accès temporaires accordés ;
— les éventuelles erreurs rencontrées lors du provisionnement.

Les événements sont transmis à Splunk à travers son **HTTP Event Collector (HEC)**. Ce mécanisme fournit au backend une interface HTTP simple permettant d'envoyer directement les événements de sécurité.

Dans l'architecture retenue, Splunk n'intervient pas dans la décision d'autorisation. Son rôle est indépendant et consiste principalement à assurer la traçabilité, l'audit et la supervision des actions réalisées.

3.8 Docker et Docker Compose — Conteneurisation

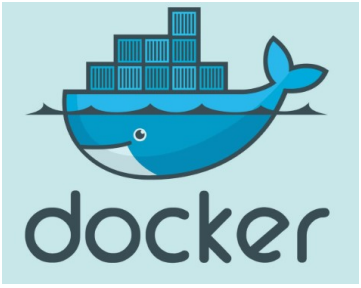


Figure 3.8 – Logo de Docker

Docker est utilisé pour déployer plusieurs composants d'infrastructure du projet dans des conteneurs isolés.

L'environnement comprend notamment des services pour :

— Keycloak ;
— PostgreSQL ;
— Open Policy Agent ;
— Splunk.

Docker Compose permet de décrire et de démarrer l'ensemble de ces services à partir du fichier `docker-compose.yml`. Il simplifie également la configuration des ports, des variables d'environnement, des volumes et du réseau reliant les conteneurs.

La conteneurisation facilite ainsi le déploiement de l'environnement de développement et évite l'installation manuelle de chaque service sur la machine hôte.

3.9 HTML, CSS et JavaScript — Interface utilisateur



Figure 3.9 – Technologies utilisées pour l'interface Web

L'interface utilisateur est développée en **HTML**, **CSS** et **JavaScript**. Elle permet à l'utilisateur d'interagir avec les différentes étapes du workflow Just-In-Time.

L'interface prend notamment en charge :

— l'authentification de l'utilisateur ;
— la saisie de la demande d'accès ;
— la saisie du ticket et de la durée souhaitée ;
— l'étape d'authentification renforcée ;
— l'affichage du résultat de la demande.

Le choix d'une interface légère en JavaScript natif permet de limiter la complexité du frontend et de conserver une visibilité directe sur les échanges entre le navigateur, Keycloak et le backend.

L'intégration avec Keycloak côté navigateur repose sur la bibliothèque **keycloak-js**, utilisée pour gérer l'authentification OpenID Connect.

3.10 PowerShell et agent local — Distribution des accès



Figure 3.10 – Logo de PowerShell

La distribution des identifiants AWS temporaires constitue une étape sensible du workflow. Afin d'éviter leur affichage direct dans l'interface Web, le projet utilise un **agent local** exécuté sur la machine de l'utilisateur.

Après validation de la demande, l'agent récupère les identifiants temporaires auprès du backend puis les injecte dans l'environnement d'une session **PowerShell** dédiée sous la forme de variables :

— `AWS_ACCESS_KEY_ID` ;
— `AWS_SECRET_ACCESS_KEY` ;
— `AWS_SESSION_TOKEN`.

L'utilisateur peut ensuite exécuter les commandes **AWS CLI** correspondant aux permissions du rôle temporaire obtenu.

Cette approche évite une manipulation manuelle des identifiants et limite leur exposition. Leur validité reste, dans tous les cas, contrôlée par la durée de la session AWS STS.

3.11 TLS et certificats auto-signés — Sécurité du transport



Figure 3.11 – Sécurisation des communications par TLS

Les communications transportant des informations sensibles doivent être protégées contre l'interception. Le projet utilise donc **HTTPS/TLS** pour chiffrer les échanges concernés entre les différents composants applicatifs.

Dans l'environnement local de développement, des certificats auto-signés sont utilisés. Ils permettent d'établir des connexions HTTPS et de vérifier le fonctionnement du chiffrement sans nécessiter une autorité de certification publique.

Cette solution est adaptée à un environnement de test contrôlé. Dans un environnement de production, les certificats devraient être délivrés par une autorité de certification de confiance ou par une infrastructure PKI interne.

L'utilisation de TLS est particulièrement importante pour les échanges contenant des jetons d'authentification ou des informations associées aux sessions AWS temporaires.

Conclusion

Le tableau 3.1 résume les principales technologies utilisées dans la solution et leur rôle.

Fonction	Technologie(s)
Gestion des identités	Keycloak, OpenID Connect
Décision d'autorisation	Open Policy Agent, Rego
Justification des demandes	Jira, API REST
Backend applicatif	Python, FastAPI, httpx, Uvicorn
Gestion des accès Cloud	AWS IAM, AWS STS, boto3
Authentification renforcée	TOTP, pyotp, Google Authenticator
Journalisation et supervision	Splunk, HTTP Event Collector
Conteneurisation	Docker, Docker Compose, PostgreSQL
Interface utilisateur	HTML, CSS, JavaScript, keycloak-js
Distribution des accès	Agent local, PowerShell, AWS CLI
Sécurité du transport	HTTPS/TLS, certificats auto-signés

Table 3.1 – Synthèse des technologies adoptées

L'architecture retenue repose ainsi sur l'intégration de plusieurs composants spécialisés. Keycloak assure l'identité, OPA prend en charge la décision d'autorisation, Jira apporte la justification métier, AWS STS fournit les privilèges temporaires et Splunk assure leur traçabilité. Le backend FastAPI orchestre ces différents composants tandis que l'agent local permet de distribuer les accès temporaires de manière contrôlée.

Le chapitre suivant présente l'architecture globale de la solution et détaille les interactions entre ces différentes technologies.

Chapitre 4

Conception et Fonctionnement de plateforme de gestion des accès Just-In-Time

Introduction

Ce chapitre présente la conception générale de la plateforme de gestion des accès privilégiés Just-In-Time développée dans le cadre de ce projet. Il décrit l'architecture globale du système, les différents composants qui interviennent dans le workflow, ainsi que les principales étapes permettant de passer d'une demande d'accès à l'obtention d'identifiants AWS temporaires.

4.1 Architecture globale de la solution

L'architecture proposée repose sur une orchestration de plusieurs composants spécialisés intervenant successivement dans le processus de gestion d'un accès temporaire.

L'utilisateur commence par créer un ticket Jira servant de justification, puis s'authentifie auprès de Keycloak. La demande d'accès est ensuite transmise à l'application de gestion des accès temporaires, qui vérifie les informations du ticket et soumet le contexte de la demande à Open Policy Agent (OPA).

Lorsque la décision d'autorisation est positive, une authentification multifacteur est exigée avant l'appel à AWS Security Token Service (STS). AWS STS génère alors des identifiants temporaires associés au rôle IAM demandé. Ces identifiants sont ensuite récupérés par un agent local et injectés dans une session PowerShell dédiée permettant l'utilisation de l'AWS CLI.

Enfin, les principaux événements du workflow sont transmis à Splunk afin d'assurer leur journalisation, leur traçabilité et leur supervision.

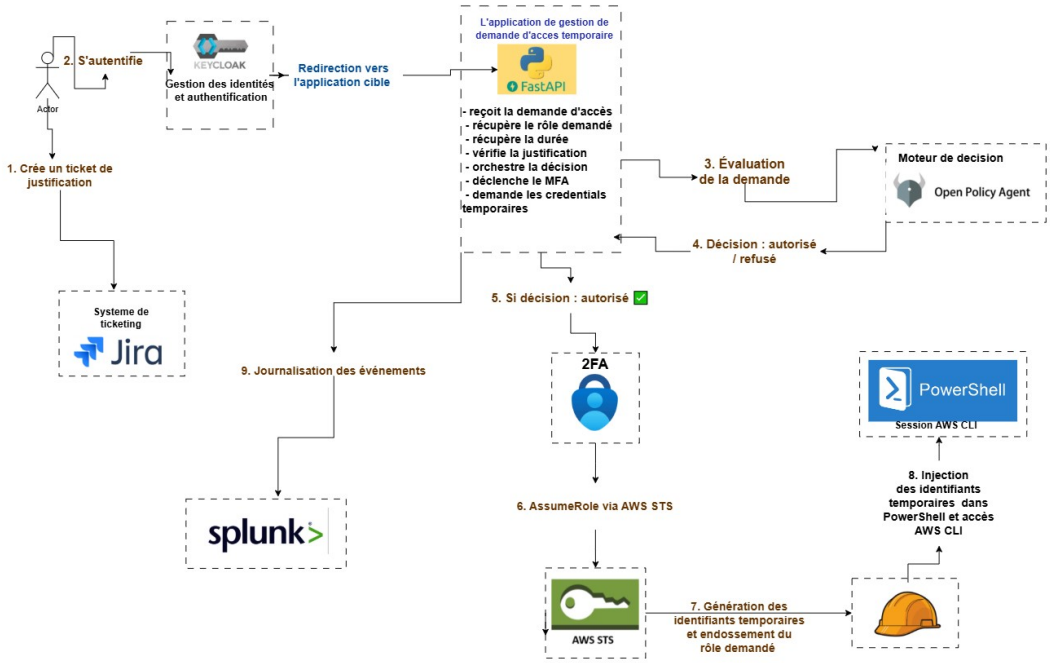


Figure 4.1 – Architecture globale de la plateforme de gestion des accès temporaires Just In Time

4.2 Déroulement global du workflow

Le fonctionnement général de la solution peut être résumé selon les étapes suivantes :

1. création d'un ticket Jira justifiant le besoin d'accès ;
2. authentification de l'utilisateur via Keycloak ;
3. soumission de la demande d'accès temporaire ;
4. vérification du ticket Jira ;
5. évaluation de la demande par OPA ;
6. authentification multifacteur ;
7. appel à AWS STS via `AssumeRole` ;
8. génération des identifiants temporaires ;
9. récupération sécurisée par l'agent local ;
10. injection des identifiants dans une session PowerShell ;
11. utilisation de l'AWS CLI avec les permissions temporaires obtenues ;
12. journalisation des événements dans Splunk.

4.3 Mise en œuvre des composants techniques

Cette section détaille la configuration et l'implémentation concrète de chacun des composants introduits précédemment, en suivant l'ordre du workflow Just-In-Time. La

mise en place de Keycloak, brique fondatrice sur laquelle reposent l'authentification et la transmission du rôle métier de l'utilisateur, est présentée en premier.

4.3.1 Configuration de Keycloak

Keycloak assure l'authentification des utilisateurs et la délivrance des jetons exploités par l'ensemble de la chaîne applicative. Sa configuration repose sur la création d'un *realm* dédié, de deux clients distincts correspondant respectivement au frontend et au backend, ainsi que des rôles métier utilisés ultérieurement par les politiques OPA.

Création du realm

Un realm nommé `jit-realm` a été créé afin d'isoler les utilisateurs, clients et rôles du projet des realms par défaut de Keycloak. Ce realm constitue le périmètre d'authentification de la plateforme.

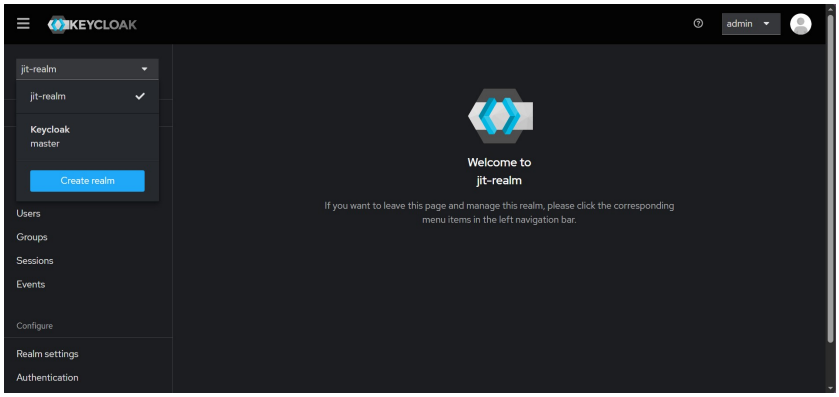


Figure 4.2 – Création du realm `jit-realm` dans la console d'administration Keycloak

Création des clients

Deux clients OpenID Connect ont été configurés au sein du realm, chacun correspondant à un profil de sécurité différent :

— `jit-frontend` : client *public*, ne pouvant conserver aucun secret confidentiel puisqu'il s'exécute entièrement côté navigateur. Le flux *Authorization Code* avec **PKCE** (`S256`) est activé, conformément à la recommandation pour les applications publiques présentée en §2.2.

— `jit-backend` : client *confidentiel*, utilisé côté serveur pour interroger le point de terminaison JWKS de Keycloak et valider la signature des jetons reçus.

Paramètre	jit-frontend	jit-backend
Type de client	Public	Confidentiel
Flow activé	Authorization Code + PKCE	Non applicable (validation JWKS)
Redirect URI	`https://127.0.0.1:5500/*`	—
Usage	Authentification navigateur	Vérification des tokens

Table 4.1 – Configuration des deux clients Keycloak du projet

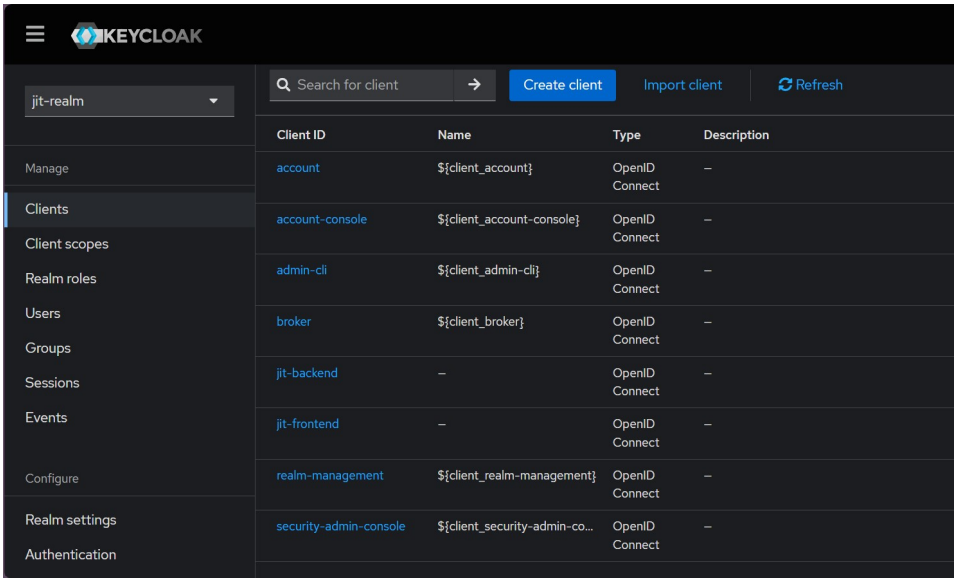


Figure 4.3 – Clients `jit-frontend` et `jit-backend` configurés dans Keycloak

Définition des rôles métier

Afin de permettre à OPA d'évaluer les demandes d'accès en fonction du profil de l'utilisateur (voir §2.3), plusieurs rôles métier ont été créés au niveau du realm :

- `devops-engineer`
- `backend-developer`
- `system-administrator`
- `qa-engineer`
- `security-analyst`
- `hr-specialist`

Chaque rôle est ensuite associé à un ou plusieurs utilisateurs de test créés dans le realm, permettant de simuler différents profils lors des tests du workflow. Ces rôles sont

automatiquement inclus dans le jeton d'accès émis par Keycloak, sous la revendication (*claim*) `realm_access.roles`.

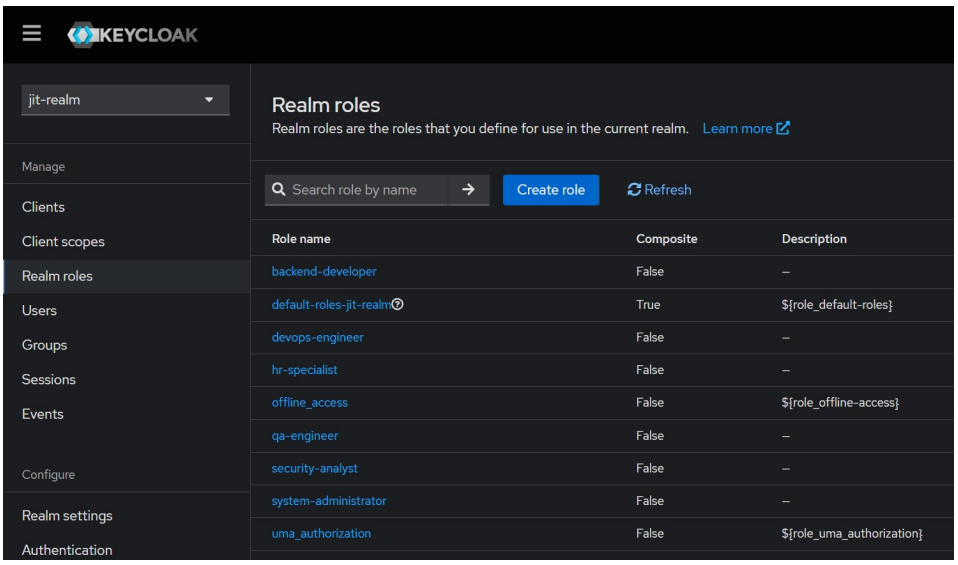


Figure 4.4 – Rôles métier définis dans le realm `jit-realm`

Authentification côté frontend

Le frontend utilise la bibliothèque **keycloak-js** afin d'initialiser une session OpenID Connect avec PKCE. Un extrait de la configuration utilisée est présenté ci-dessous :

Listing 4.1 – Configuration du client Keycloak côté frontend

```
const KEYCLOAK_CONFIG = {
  url: "http://localhost:8080",
  realm: "jit-realm",
  clientId: "jit-frontend",
};
```

Lors du chargement de l'application, la bibliothèque vérifie automatiquement l'existence d'une session active (`check-sso`) et déclenche, le cas échéant, la redirection vers la page de connexion Keycloak. Un rafraîchissement périodique du jeton est également mis en place afin d'éviter son expiration au cours d'une session utilisateur.

Validation des jetons côté backend

Le backend ne gère jamais directement les identifiants de l'utilisateur : il reçoit uniquement le jeton d'accès déjà émis par Keycloak et en vérifie l'authenticité. Cette vérification repose sur la récupération des clés publiques exposées via le point de terminaison JWKS de Keycloak :

Listing 4.2 – Point de terminaison JWKS interrogé par le backend

```
GET /realms/jit-realm/protocol/openid-connect/certs
```

La signature du jeton est vérifiée localement à partir de ces clés (algorithme `RS256`), sans nécessiter de nouvel appel réseau vers Keycloak à chaque requête, conformément au principe d'auto-suffisance des jetons JWT présenté en §2.2.4. Une fois le jeton validé, les informations utiles — notamment l'adresse électronique de l'utilisateur et ses rôles métier — sont extraites et transmises aux étapes suivantes du workflow, en particulier à l'évaluation par OPA.

Point clé. Le backend n'authentifie jamais lui-même l'utilisateur et ne stocke aucun mot de passe : il se limite à vérifier l'intégrité et la validité d'un jeton déjà émis par un tiers de confiance (Keycloak), ce qui respecte la séparation stricte entre authentification et autorisation évoquée en §2.1.

4.3.2 Mise en place d'Open Policy Agent (OPA) et des politiques Rego

Une fois l'utilisateur authentifié et sa demande transmise au backend, la décision d'autoriser ou non l'accès temporaire est entièrement déléguée à **Open Policy Agent (OPA)**, conformément au principe de *policy-as-code* présenté en §2.3. Le backend ne contient ainsi aucune logique métier d'autorisation : il se limite à transmettre un contexte structuré à OPA et à appliquer la décision reçue.

Déploiement du service OPA

OPA est déployé sous forme de conteneur Docker, démarré en mode serveur et exposé sur le port `8181`. Le répertoire local contenant les fichiers de politique est monté dans le conteneur, ce qui permet de modifier les règles Rego sans reconstruire l'image :

Listing 4.3 – Extrait du service OPA dans docker-compose.yml

```
opa:
  image: openpolicyagent/opa:latest
  command: ["run", "--server", "--addr=:8181", "/policies"]
  ports:
    - "8181:8181"
  volumes:
    - ./opa/policies:/policies
```

Figure 4.5 – Conteneur OPA actif, exposant le port 8181

Interrogation d'OPA depuis le backend

Le backend interroge OPA via son API HTTP, en soumettant l'ensemble du contexte de la demande (utilisateur, requête, ticket, horodatage courant) à l'endpoint correspondant à la règle `allow` du package `jit.authz` :

Listing 4.4 – Endpoint OPA interrogé par le backend

```
POST /v1/data/jit/authz/allow
{ "input": { "user": {...}, "request": {...}, "ticket": {...},
   "context": {...} } }
```

La réponse d'OPA est un booléen (`result`) que le backend applique directement, sans réévaluation locale. Cette séparation garantit que toute modification des règles de sécurité s'effectue exclusivement au niveau des fichiers Rego, sans nécessiter de modification ni de redéploiement du backend.

Structure de la politique Rego

La politique est définie dans le package `jit.authz`. La règle `allow` n'est satisfaite que si l'ensemble des conditions suivantes sont simultanément vraies :

Listing 4.5 – Règle principale d'autorisation

```
package jit.authz

default allow := false

allow if {
    role_can_request_this
    ticket_is_active
    ticket_assigned_to_requester
    priority_is_sufficient
    duration_within_limit
    duration_above_minimum
    is_within_business_hours
}
```

Le fait de fixer `default allow := false` garantit qu'en l'absence de correspondance explicite avec l'ensemble des conditions, la demande est **refusée par défaut** — une application directe du principe de *fail-safe default* attendu de tout mécanisme d'autorisation.

Matrice de correspondance rôle métier / rôle demandé

La première condition repose sur une matrice associant chaque rôle métier Keycloak (voir §4.3.1) à l'ensemble des rôles temporaires qu'il est autorisé à solliciter :

Rôle métier (Keycloak)	Rôles temporaires autorisés
`devops-engineer`	`s3-write-access`, `s3-read-access`
`backend-developer`	`s3-write-access`, `s3-read-access`
`system-administrator`	
`qa-engineer`	`s3-read-access`
`security-analyst`	`s3-write-access`, `s3-read-access`
`hr-specialist`	aucun (ensemble vide)

Table 4.2 – Matrice de correspondance entre rôles métier et rôles temporaires

Cette matrice constitue une application concrète du modèle **ABAC** présenté en §2.3 : la décision ne dépend pas uniquement d'un rôle, mais de la combinaison de plusieurs attributs, dont fait partie le rôle métier de l'utilisateur.

Vérifications liées au ticket Jira

Deux conditions portent sur le ticket fourni en justification. Le ticket doit d'abord se trouver dans un statut actif :

Listing 4.6 – Vérification du statut du ticket

```
ticket_is_active if { input.ticket.status == "A faire" }
ticket_is_active if { input.ticket.status == "En cours" }
```

Il doit ensuite être explicitement assigné à l'utilisateur effectuant la demande, ce qui empêche un utilisateur d'invoquer un ticket créé ou assigné à un tiers :

Listing 4.7 – Vérification de l'assignation du ticket

```
ticket_assigned_to_requester if {
    input.ticket.assignee == input.user.email
}
```

Enfin, la priorité du ticket doit être suffisante pour justifier l'élévation de privilèges demandée. Les priorités `Highest`, `High` et `Medium` sont acceptées, à l'exclusion des tickets de priorité `Low` ou `Lowest`.

Contraintes de durée

La durée d'accès demandée est encadrée par deux bornes. Une durée minimale de 15 minutes est imposée, cette valeur correspondant à la durée minimale techniquement acceptée par AWS STS pour une session temporaire (voir §2.4). Une durée maximale de

240 minutes (4 heures) est également appliquée, afin de limiter la fenêtre d'exposition d'un accès privilégié, conformément au principe du moindre privilège présenté en §2.1.

Restriction temporelle de la demande

Une dernière condition restreint les horaires durant lesquels une demande peut être acceptée, en s'appuyant sur les fonctions temporelles natives de Rego afin de convertir l'horodatage de la requête dans le fuseau horaire de Casablanca :

Listing 4.8 – Vérification de la plage horaire autorisée

```
is_within_business_hours if {
    ts_ns := time.parse_rfc3339_ns(input.context.current_time)
    clock := time.clock([ts_ns, "Africa/Casablanca"])
    clock[0] >= 9
}

is_within_business_hours if {
    ts_ns := time.parse_rfc3339_ns(input.context.current_time)
    clock := time.clock([ts_ns, "Africa/Casablanca"])
    clock[0] < 17
}
```

Traçabilité des décisions

Chaque décision, qu'elle soit positive ou négative, est ensuite relayée par le backend vers Splunk, avec le détail du contexte de la demande. Ce couplage permet de conserver une justification complète pour chaque octroi ou refus, y compris lorsque la demande échoue dès l'étape d'évaluation par OPA, avant même que l'authentification multifacteur ne soit sollicitée.

Point clé. L'ensemble des règles présentées ci-dessus est combiné par un opérateur logique *ET* implicite au sein de la règle `allow` : la moindre condition non satisfaite entraîne le refus de la demande. Cette conception *fail-closed* garantit qu'aucune combinaison imprévue de circonstances ne peut aboutir à un octroi de privilèges non anticipé par la politique.

4.3.3 Intégration avec Jira

Dans la plateforme développée, Jira constitue la source de justification de la demande d'accès temporaire. Avant de pouvoir solliciter un rôle AWS, l'utilisateur doit disposer d'un ticket Jira valide correspondant au besoin opérationnel à l'origine de la demande. Le backend interroge ainsi Jira afin de récupérer les informations associées au ticket avant

de transmettre celles-ci à Open Policy Agent (OPA) pour l'évaluation de la politique d'autorisation.

Domaine Jira

L'intégration repose sur une instance Jira Cloud accessible à travers un **domaine Atlassian** propre à l'organisation, de la forme `nom-instance.atlassian.net`. Cet espace centralise les projets, les utilisateurs et les tickets utilisés dans le cadre de la plateforme.Dans notre cas, le nom de domaine code212 s'avere appropriee.



Figure 4.6 – domaine Atlassian

Espace Jira dédié au projet

Un espace Jira nommé `JIT-Access` a été créé afin de centraliser les tickets associés aux demandes d'accès temporaire. Cet espace permet de regrouper les différentes demandes, de suivre leur état d'avancement et d'identifier les utilisateurs concernés. Chaque ticket possède une clé unique, telle que `JIT-1`, qui est utilisée par l'utilisateur lors de la soumission de sa demande d'accès au sein de la plateforme. Cette clé permet ensuite au backend d'interroger Jira afin de récupérer les informations nécessaires à l'évaluation de la demande.

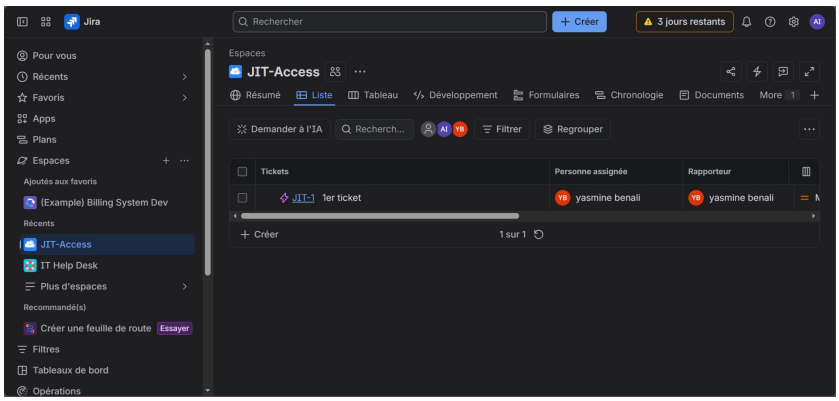


Figure 4.7 – Espace Jira `JIT-Access` dédié à la gestion des tickets de demande d'accès temporaire

Configuration de l'accès à l'API Jira

L'intégration avec Jira repose sur son API REST. Une authentification dédiée est utilisée par le backend afin de pouvoir consulter les tickets sans nécessiter l'intervention manuelle de l'utilisateur. La création de jeton se fait en accedant a l'URL suivante :

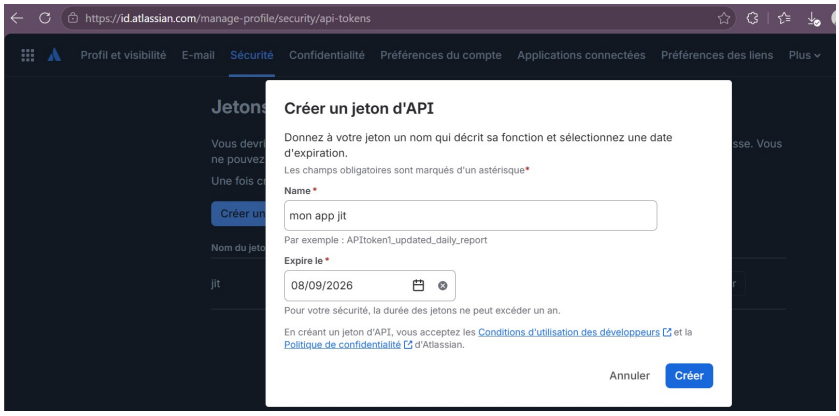


Figure 4.8 – Clé API de Jira

Les paramètres nécessaires à la communication avec Jira sont stockés dans la configuration du backend et ne sont jamais exposés côté frontend. Ils comprennent notamment l'URL de l'instance Jira, l'identifiant du compte utilisé pour les appels API ainsi que le jeton d'accès associé.

Listing 4.9 – Paramètres de connexion à Jira utilisés par le backend

```
JIRA_BASE_URL=https://code212.atlassian.net
JIRA_EMAIL=<service-account-email>
JIRA_API_TOKEN=<api-token>
```

Cette séparation permet d'éviter l'exposition des informations d'authentification Jira dans le navigateur et centralise l'ensemble des échanges avec le service au niveau du backend.

Soumission du numéro de ticket

Lors de la création d'une demande d'accès temporaire, l'utilisateur renseigne le numéro du ticket Jira associé à son besoin, par exemple `JIT-1`. Cette référence est transmise au backend avec les autres paramètres de la demande, notamment le rôle temporaire souhaité et la durée d'accès demandée.

Le backend construit ensuite une requête vers l'API Jira afin de récupérer les informations du ticket correspondant :

Listing 4.10 – Requête de récupération d'un ticket Jira

```
GET /rest/api/3/issue/JIT-1
```

La réponse fournie par Jira contient l'ensemble des informations associées au ticket. Seuls les attributs nécessaires à la décision d'autorisation sont conservés par l'application.

Extraction des attributs utiles

Afin de limiter les données manipulées par la plateforme, le backend extrait uniquement les informations utilisées par les politiques de contrôle d'accès. Dans le cadre de ce projet, les principaux attributs exploités sont :

— la clé du ticket ;
— le statut courant du ticket ;
— la priorité ;
— l'utilisateur auquel le ticket est assigné.

Ces informations sont normalisées avant d'être intégrées au contexte envoyé à OPA. Le backend peut ainsi construire une représentation simplifiée du ticket :

Listing 4.11 – Structure du ticket transmise à OPA

```
{
"ticket": {
"key": "JIT-243",
"status": "En cours",
"priority": "High",
"assignee": "aya.alami@gmail.com]"
}
}
```

Cette normalisation découple la politique Rego de la structure complète retournée par l'API Jira et permet de ne transmettre à OPA que les attributs strictement nécessaires à la prise de décision.

Vérification de l'association entre ticket et utilisateur

L'un des contrôles essentiels consiste à vérifier que le ticket fourni appartient effectivement au contexte de l'utilisateur authentifié. L'adresse électronique extraite du jeton Keycloak est comparée à celle de l'utilisateur auquel le ticket Jira est assigné.

Ainsi, un utilisateur ne peut pas utiliser arbitrairement le ticket d'un autre collaborateur afin de justifier une demande d'élévation de privilèges.

Gestion des tickets invalides

Lorsque le numéro renseigné ne correspond à aucun ticket, lorsque Jira ne peut pas être contacté ou lorsque les informations nécessaires ne peuvent pas être récupérées, la demande n'est pas poursuivie.

Le backend adopte ainsi un comportement *fail-closed* : l'indisponibilité de la source permettant de justifier l'accès ne doit jamais entraîner un octroi de privilèges par défaut.

Les erreurs liées à Jira sont également journalisées afin de permettre leur consultation ultérieure dans Splunk.

Point clé. Jira ne prend aucune décision d'autorisation dans la plateforme. Il fournit uniquement le contexte métier justifiant la demande. La décision reste entièrement prise par OPA à partir des informations de l'utilisateur, de la demande et du ticket récupéré.

4.3.4 Authentification multifacteur par TOTP

Lorsqu'une demande satisfait l'ensemble des règles d'autorisation définies dans OPA, une seconde vérification de l'identité de l'utilisateur est imposée avant toute génération d'identifiants AWS temporaires. Cette étape repose sur une authentification multifacteur utilisant le mécanisme **TOTP** (*Time-Based One-Time Password*).

L'objectif est de garantir que la connaissance des seuls identifiants de connexion Keycloak ne suffit pas pour obtenir un accès privilégié. L'utilisateur doit également prouver qu'il possède le facteur supplémentaire préalablement associé à son compte.

Principe du mécanisme TOTP

Le mécanisme TOTP repose sur la génération de mots de passe à usage unique dont la validité est limitée dans le temps. Lors de l'enrôlement, un secret partagé est associé au compte de l'utilisateur et enregistré dans une application d'authentification compatible, telle qu'une application TOTP sur smartphone.

À partir de ce secret et de l'heure courante, l'application génère périodiquement un code temporaire. Le même calcul est effectué côté serveur afin de vérifier le code fourni par l'utilisateur.

Un code expiré ne peut donc pas être réutilisé lors d'une nouvelle demande.

Enrôlement du second facteur

Lors de la première configuration de l'authentification multifacteur, l'utilisateur associe son application d'authentification à son compte. Un secret TOTP est généré puis représenté sous la forme d'un QR Code pouvant être scanné depuis l'application mobile.

Une fois cet enrôlement terminé, les futurs codes temporaires peuvent être générés directement par l'application d'authentification, sans nécessiter d'échange permanent avec celle-ci.

Déclenchement de la MFA dans le workflow JIT

La MFA n'est demandée qu'après validation de la politique d'autorisation par OPA. Cet ordre permet d'éviter de solliciter inutilement un second facteur pour une demande qui serait de toute manière refusée par les règles métier.

Le workflow suivi est ainsi le suivant :

1. le backend reçoit la demande d'accès ;
2. le ticket Jira est vérifié ;
3. le contexte est évalué par OPA ;
4. OPA retourne une décision positive ;
5. l'utilisateur est invité à saisir son code TOTP ;
6. le code est vérifié ;
7. en cas de succès, le workflow peut poursuivre vers AWS STS.

Validation du code TOTP

Après la décision positive d'OPA, l'interface présente à l'utilisateur un champ permettant de saisir le code temporaire affiché dans son application d'authentification.

Le code est transmis au backend via la connexion HTTPS de l'application. Le backend vérifie ensuite sa validité avant d'autoriser le passage à l'étape suivante.

La réponse peut être représentée de manière simplifiée par les deux états suivants :

Listing 4.12 – Résultat logique de la vérification TOTP

```
MFA valide   -> poursuite du workflow
MFA invalide -> refus de la demande
```

Aucun appel à AWS STS n'est effectué tant que cette vérification n'a pas abouti avec succès.

Traitement d'un échec d'authentification

Lorsqu'un code incorrect ou expiré est soumis, la génération des identifiants temporaires est immédiatement bloquée. L'événement est ensuite journalisé afin de conserver une trace de l'échec de l'authentification multifacteur.

Cette mesure permet notamment d'identifier des tentatives répétées d'utilisation d'un compte dont le premier facteur aurait éventuellement été compromis.

La journalisation contient uniquement les informations nécessaires à l'audit, telles que l'identité de l'utilisateur, la référence de la demande, l'horodatage et le résultat de l'authentification. Le code TOTP lui-même n'est pas enregistré.

Passage vers AWS STS

Une validation TOTP réussie constitue la dernière étape de vérification avant l'octroi effectif des privilèges temporaires.

Le backend dispose alors de trois garanties complémentaires :

— l'utilisateur a été authentifié par Keycloak ;
— la demande satisfait les politiques d'autorisation définies dans OPA ;
— l'utilisateur a validé le second facteur d'authentification TOTP.

Ce n'est qu'après la satisfaction de ces trois conditions que le backend est autorisé à appeler AWS Security Token Service afin d'assumer le rôle IAM correspondant à la demande et de générer les identifiants temporaires.

Point clé. Le TOTP ne remplace ni l'authentification initiale par Keycloak ni l'autorisation par OPA. Il constitue une barrière supplémentaire avant l'accès privilégié et assure qu'une compromission du premier facteur d'authentification ne suffit pas, à elle seule, pour déclencher la génération d'identifiants AWS temporaires.

4.4 Mise en œuvre des accès temporaires avec AWS IAM et STS

Afin de permettre à l'application de générer des accès temporaires vers les ressources AWS, une configuration spécifique a été mise en place au niveau de **AWS IAM** et de **AWS Security Token Service (STS)**.

Le principe retenu consiste à associer le backend à une identité IAM dédiée, disposant uniquement du droit d'assumer des rôles précis. Les permissions d'accès aux ressources ne sont donc pas directement attribuées à cet utilisateur IAM : elles sont portées par des rôles temporaires distincts.

4.4.1 Création de l'utilisateur IAM applicatif

Un utilisateur IAM nommé `mon-app-jit` a été créé afin de représenter l'application auprès d'AWS.

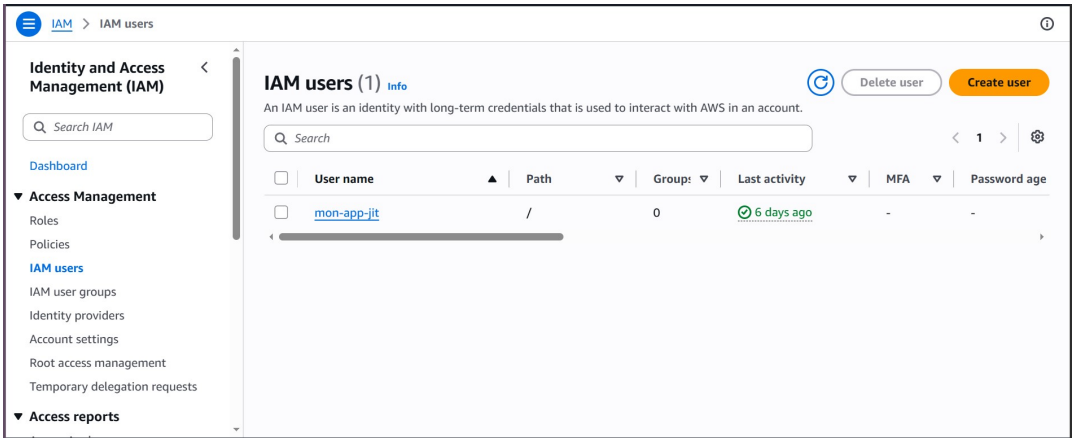


Figure 4.9 – Utilisateur IAM dédié à l'application JIT

Cet utilisateur constitue l'identité utilisée par le backend pour effectuer les appels vers AWS STS. Il ne dispose pas directement de permissions de lecture ou d'écriture sur le bucket S3.

4.4.2 Autorisation d'assumer les rôles

Une politique IAM nommée `s3AssumeRoles` a été associée à l'utilisateur `mon-app-jit`.

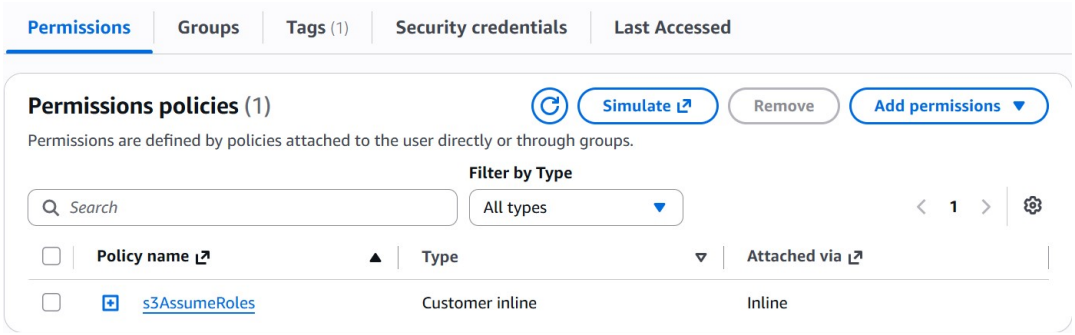


Figure 4.10 – Politique d'autorisation associée à l'utilisateur IAM

Cette politique autorise uniquement l'action :

`sts:AssumeRole`

sur deux rôles précis :

— `s3ReadRole` ;
— `s3WriteRole`.

La politique correspondante est de la forme suivante :

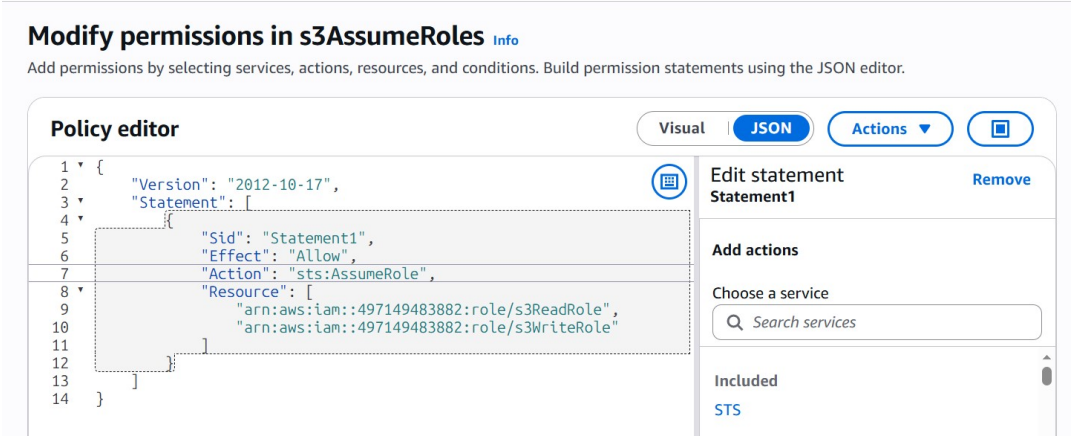


Figure 4.11 – Politique d'autorisation détaillée associée à l'utilisateur IAM

Cette configuration applique le principe du moindre privilège : le backend ne peut assumer que les rôles explicitement autorisés.

4.4.3 Création des rôles IAM

Deux rôles IAM ont ensuite été créés pour représenter les deux niveaux d'accès utilisés dans le projet :

— `s3ReadRole`, destiné aux accès en lecture ;
— `s3WriteRole`, destiné aux accès en écriture.

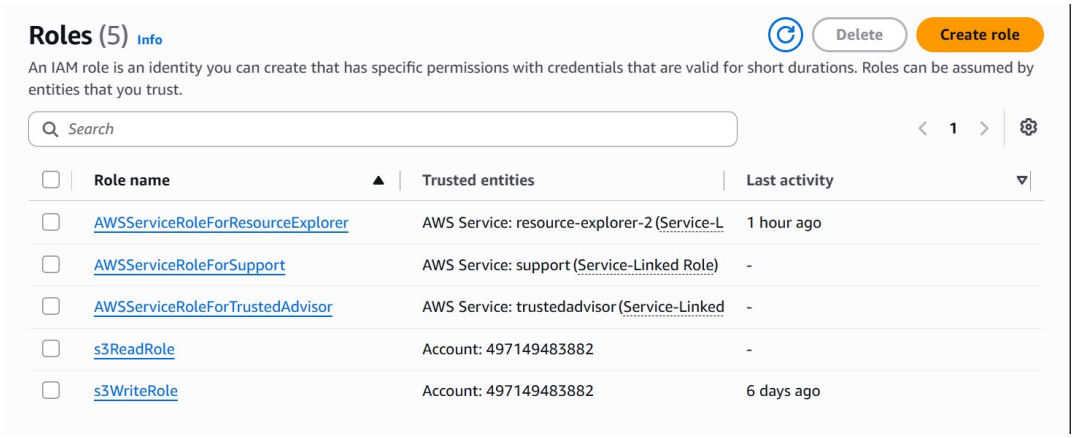


Figure 4.12 – Rôles IAM utilisés pour les accès temporaires

Chaque rôle possède deux éléments essentiels : une **trust policy**, qui détermine qui peut assumer le rôle, et une **permissions policy**, qui définit les actions autorisées une fois le rôle assumé.

4.4.4 Configuration de la relation de confiance

La relation de confiance des rôles autorise l'utilisateur `mon-app-jit` à effectuer l'opération `sts:AssumeRole`.

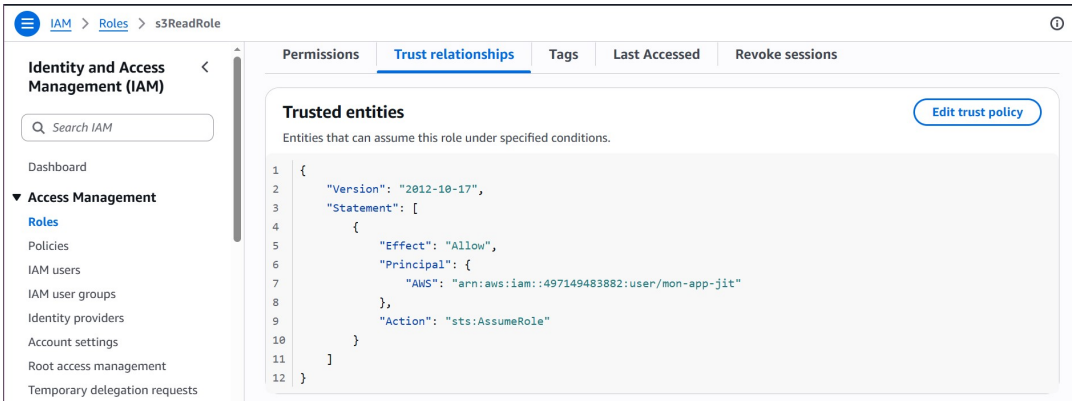


Figure 4.13 – Relation de confiance du rôle `s3ReadRole`

Ainsi, l'autorisation fonctionne dans les deux sens :

— l'utilisateur `mon-app-jit` possède le droit d'appeler `AssumeRole` ;
— le rôle accepte cet utilisateur comme entité de confiance.

4.4.5 Création des politiques de permissions S3

Deux politiques de permissions ont été définies afin de séparer les privilèges de lecture et d'écriture :

— `s3ReadPolicy` ;
— `s3WritePolicy`.

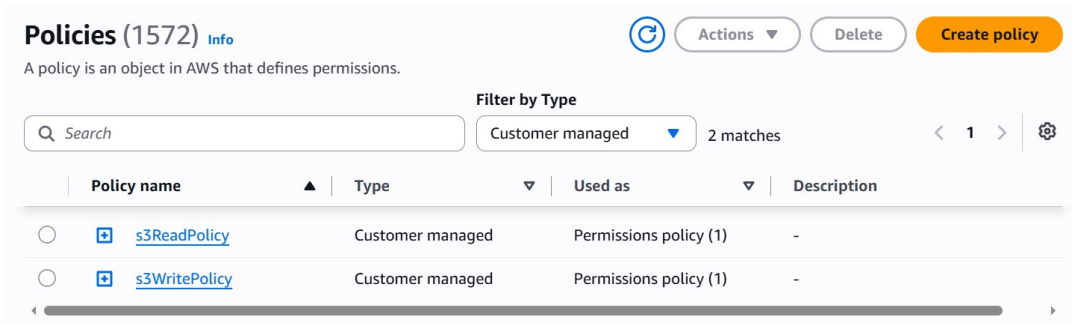


Figure 4.14 – Politiques IAM utilisées pour les accès S3

La politique `s3ReadPolicy` limite l'utilisateur temporaire aux opérations de lecture sur le bucket cible.

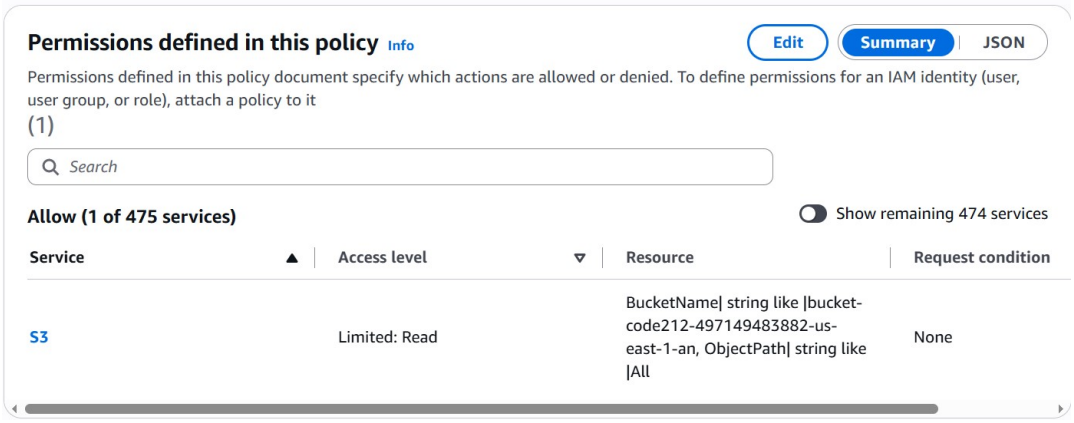


Figure 4.15 – Permissions de lecture définies par `s3ReadPolicy`

La politique `s3WritePolicy` fournit quant à elle les permissions nécessaires aux opérations d'écriture.

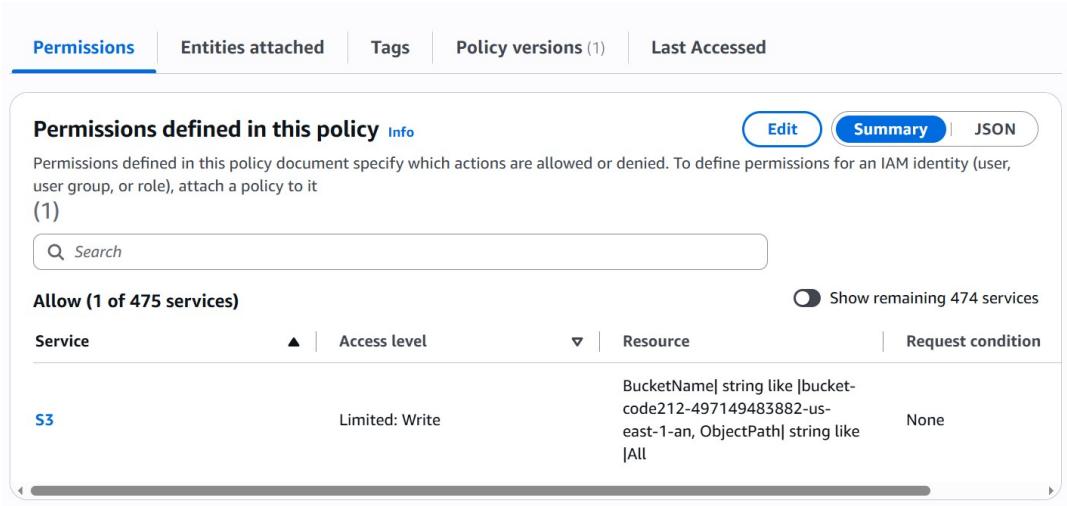


Figure 4.16 – Permissions d'écriture définies par `s3WritePolicy`

Chaque politique est ensuite rattachée au rôle correspondant. Par exemple, `s3ReadPolicy` est associée à `s3ReadRole`.

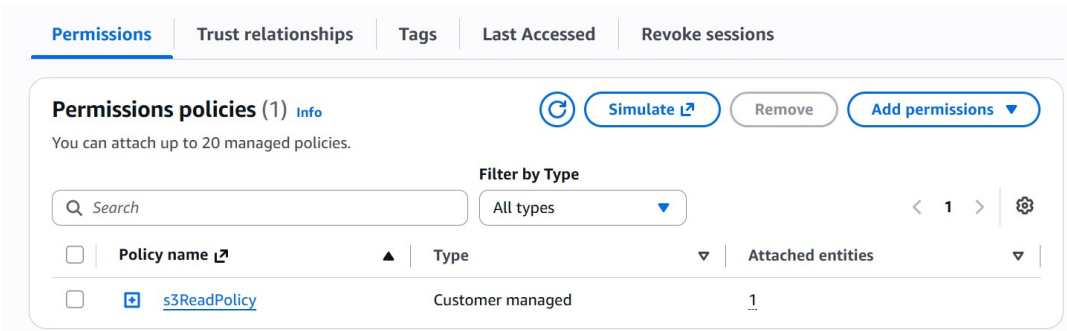


Figure 4.17 – Association de la politique `s3ReadPolicy` au rôle `s3ReadRole`

La même logique est appliquée au rôle `s3WriteRole` avec la politique `s3WritePolicy`.

4.4.6 Création de la ressource Amazon S3

Afin de tester concrètement les accès temporaires délivrés par le système, un bucket Amazon S3 a été créé dans la région `us-east-1`.

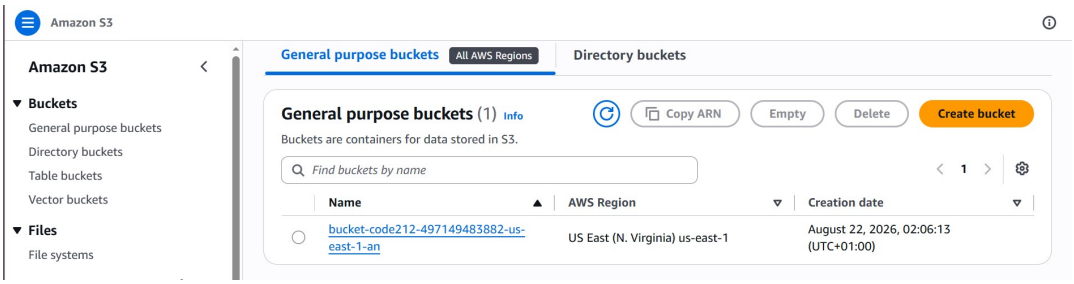


Figure 4.18 – Bucket Amazon S3 utilisé pour la validation des accès temporaires

Les politiques `s3ReadPolicy` et `s3WritePolicy` sont limitées à ce bucket, ce qui permet de tester précisément la séparation des privilèges.

4.4.7 Fonctionnement de AWS STS dans le projet

Une fois la demande JIT validée par les différentes étapes de contrôle, le backend utilise **boto3** pour appeler AWS STS avec l'opération `AssumeRole`.

Selon le rôle demandé, le backend sélectionne l'ARN correspondant :

— `s3-read-access` → `s3ReadRole` ;
— `s3-write-access` → `s3WriteRole`.

AWS STS retourne alors des identifiants temporaires composés de :

— `AccessKeyId` ;
— `SecretAccessKey` ;
— `SessionToken` ;
— une date d'expiration.

Ces identifiants héritent exclusivement des permissions du rôle assumé et deviennent automatiquement invalides après expiration. Cette architecture garantit que l'utilisateur final ne reçoit jamais de compte IAM permanent et ne dispose que des permissions nécessaires pendant la durée autorisée.

4.5 Agent local et distribution sécurisée des accès temporaires

La distribution des identifiants temporaires constitue la dernière phase du workflow Just-In-Time. Une fois la demande validée, le MFA réussi et le rôle AWS assumé via

STS, l'utilisateur doit pouvoir exploiter les privilèges accordés sans que les credentials temporaires soient directement exposés dans l'interface Web.

Pour répondre à cette exigence, la solution intègre un **agent local** exécuté sur le poste de travail de l'utilisateur. Son rôle est de récupérer de manière contrôlée les identifiants temporaires générés par AWS STS, puis de les injecter automatiquement dans une session PowerShell dédiée.

4.5.1 Principe de distribution

Après la génération des credentials temporaires, le backend ne les renvoie pas directement au navigateur. Il crée à la place un `launch_token` à durée de vie courte, destiné uniquement à déclencher l'agent local.

Le navigateur transmet ensuite ce jeton à l'agent installé sur le poste de l'utilisateur. L'agent contacte le backend pour effectuer une opération de retrait unique et récupérer les éléments suivants :

— `AWS_ACCESS_KEY_ID` ;
— `AWS_SECRET_ACCESS_KEY` ;
— `AWS_SESSION_TOKEN` ;
— la date d'expiration de la session STS.

Cette approche permet de limiter l'exposition des secrets, puisque les credentials AWS ne transitent pas directement dans l'interface Web.

4.5.2 Ouverture d'une session PowerShell dédiée

Après récupération des credentials, l'agent local ouvre automatiquement une nouvelle session **PowerShell** et injecte les identifiants AWS sous forme de variables d'environnement.

La session obtenue est alors directement utilisable avec l'AWS CLI. L'utilisateur peut exécuter uniquement les opérations autorisées par le rôle temporaire obtenu, par exemple les opérations de lecture ou d'écriture sur le bucket S3 selon le rôle assumé.

4.5.3 Utilisation d'un poste de travail légitime

Le choix d'un agent local répond également à une logique de sécurité liée au poste de travail. Dans un environnement d'entreprise, le poste sur lequel s'exécute l'agent est supposé avoir déjà satisfait les contrôles de sécurité imposés par l'organisation, par exemple son appartenance au parc informatique autorisé, sa présence sur le réseau interne ou encore sa conformité aux politiques de sécurité de l'entreprise.

L'objectif est donc de faire en sorte que les accès temporaires soient utilisés depuis une machine considérée comme légitime et contrôlée, et non depuis n'importe quel terminal disposant uniquement d'un navigateur Web.

Ainsi, l'utilisateur ne reçoit pas simplement des credentials qu'il pourrait copier et utiliser depuis un autre équipement. Les identifiants sont injectés directement dans une session locale ouverte sur le poste autorisé, ce qui permet de mieux maîtriser le contexte dans lequel les privilèges temporaires sont utilisés.

4.5.4 Expiration automatique de la session

La durée de validité de la session PowerShell est alignée sur la durée des identifiants AWS STS. Lorsque l'heure d'expiration est atteinte, l'agent ferme automatiquement la session dédiée.

Même dans le cas où cette fermeture ne serait pas réalisée, les identifiants STS deviennent automatiquement invalides côté AWS après expiration. Les permissions ne peuvent donc plus être utilisées au-delà de la durée accordée.

Cette étape garantit que l'accès privilégié reste strictement temporaire et respecte le principe du *Just-In-Time Access*.

4.5.5 Apport de l'agent local

L'utilisation de l'agent local permet ainsi de répondre à plusieurs objectifs de sécurité :

— éviter l'affichage des credentials AWS dans l'interface Web ;
— limiter leur manipulation manuelle par l'utilisateur ;
— utiliser un jeton de retrait temporaire et à usage unique ;
— injecter automatiquement les credentials dans une session dédiée ;
— privilégier l'utilisation des accès depuis un poste de travail considéré comme légitime ;
— aligner la durée de la session locale sur l'expiration des credentials AWS STS.

Cette phase constitue donc le dernier maillon du workflow JIT : elle transforme la décision d'autorisation en une session de travail temporaire, directement exploitable par l'utilisateur tout en maintenant un niveau de contrôle élevé sur la distribution et l'utilisation des privilèges.

4.6 Journalisation et supervision avec Splunk

Chaque étape significative du workflow Just-In-Time — qu'elle aboutisse à un octroi ou à un refus — est relayée vers Splunk afin d'assurer la traçabilité complète du système, conformément au rôle attribué à l'observabilité en §2.5.

Déploiement et configuration du HTTP Event Collector

Splunk est déployé sous forme de conteneur Docker au sein de l'infrastructure locale du projet. La réception des événements applicatifs repose sur son **HTTP Event Collector (HEC)**, un point de terminaison HTTP dédié à l'ingestion d'événements structurés, activé via un jeton d'authentification :

Listing 4.13 – Configuration du conteneur Splunk (extrait docker-compose.yml)

```
splunk:
  image: splunk/splunk:latest
  container_name: splunk
  environment:
    SPLUNK_HEC_TOKEN: "--------"
```

Côté backend, l'URL du collecteur et le jeton associé sont définis comme variables d'environnement, ce qui permet de reconfigurer la destination des journaux sans modifier le code source :

Listing 4.14 – Variables d'environnement du service Splunk

```
SPLUNK_HEC_URL = https://localhost:8088/services/collector/event
SPLUNK_HEC_TOKEN = --------
```

Conception non bloquante de l'envoi des événements

Un principe de conception essentiel a été appliqué à ce service : la disponibilité de Splunk ne doit **jamais** conditionner le déroulement du workflow métier, conformément au principe de non-interférence de la couche d'observabilité présenté en §2.5. L'appel HTTP vers le HEC est ainsi encapsulé dans un bloc capturant explicitement toute erreur réseau, sans jamais la propager :

Listing 4.15 – Fonction d'envoi non bloquante vers Splunk

```
def send_event(event_type: str, data: dict) -> None:
    payload = {
        "event": {"event_type": event_type,
                  "timestamp": datetime.utcnow().isoformat(), **data
                     },
        "sourcetype": "jit_access_logs",
    }
    try:
        httpx.post(SPLUNK_HEC_URL,
                  headers={"Authorization": f"Splunk {
                     SPLUNK_HEC_TOKEN}"},
                  json=payload, timeout=3.0, verify=False)
    except httpx.HTTPError:
```

```
        pass  # un Splunk indisponible ne doit jamais bloquer une
            decision legitime
```

Ainsi, même en cas de panne complète de Splunk, un utilisateur légitime continue d'obtenir ses accès temporaires ; seule la trace de l'événement correspondant est alors perdue.

Catalogue des événements journalisés

Chaque appel à `send_event` est déclenché à un point précis du backend (`main.py`), correspondant à une étape clé décrite dans les sections précédentes. Le tableau 4.3 récapitule les types d'événements générés.

Type d'événement	Déclencheur
`access_denied`	Ticket introuvable ou demande rejetée par la politique OPA
`mfa_required`	Demande autorisée par OPA, en attente de validation du code TOTP
`mfa_failed`	Code TOTP invalide fourni par l'utilisateur
`access_granted`	Provisioning AWS STS réussi, identifiants temporaires générés
`provisioning_failed`	Erreur lors de l'appel à AWS STS (`AssumeRole`)
`agent_credentials_claimed`	Retrait effectif des identifiants par l'agent local

Table 4.3 – Catalogue des événements de sécurité transmis à Splunk

Conclusion

Ce chapitre a présenté la conception et le fonctionnement de la plateforme de gestion des accès Just-In-Time. La solution combine Keycloak pour l'authentification, Jira pour la justification des demandes, OPA pour la décision d'autorisation, la MFA pour renforcer la sécurité, ainsi qu'AWS IAM et STS pour générer des accès temporaires limités dans le temps. Un agent local permet ensuite d'utiliser ces identifiants dans une session PowerShell dédiée, tandis que Splunk assure la traçabilité des événements. L'ensemble permet ainsi de mettre en œuvre un accès privilégié temporaire, contrôlé et conforme au principe du moindre privilège.

Chapitre 5

Architecture et Implémentation de l'Application Just-in-Time Access

Introduction

Après avoir présenté l'architecture fonctionnelle et les différents mécanismes de la plateforme Just-In-Time, ce chapitre se concentre sur son implémentation applicative. L'objectif est de présenter l'organisation du projet, la répartition des responsabilités entre ses différents modules ainsi que le rôle des principaux fichiers constituant l'application.

L'application est organisée selon une architecture modulaire séparant l'interface utilisateur, l'orchestration du workflow, les services d'intégration, les politiques d'autorisation, l'agent local et l'infrastructure conteneurisée. Cette séparation facilite la maintenance du projet et permet à chaque composant d'assurer une responsabilité précise.

5.1 Architecture du projet

L'organisation générale de l'application est présentée par l'arborescence suivante :

```
jit-access/
|
|-- backend/
|   |-- main.py
|   |-- auth.py
|   |-- models.py
|   |-- session_store.py
|   |-- requirements.txt
|   |
|   `-- services/
|       |-- jira_service.py
|       |-- opa_service.py
|       |-- mfa_service.py
|       |-- aws_service.py
|       `-- splunk_service.py
|
```

```
|-- frontend/
|   |-- index.html
|   |-- style.css
|   |-- config.js
|   `-- app.js
|
|-- opa/
|   `-- policies/
|       `-- jit_authz.rego
|
|-- certs/
|   |-- localhost.crt
|   `-- localhost.key
|
|-- scripts/
|   `-- generate_local_cert.py
|
|-- agent_jit.py
|-- docker-compose.yml
|-- serve_frontend_https.py
|-- start_backend_https.ps1
|-- start_frontend_https.ps1
`-- start_agent.ps1
```

Cette architecture peut être regroupée en cinq parties principales : le frontend, le backend, les services d'intégration, le moteur de politiques OPA et l'agent local. Docker Compose assure en parallèle le déploiement des services d'infrastructure.

5.2 Interface utilisateur

Le répertoire `frontend` contient l'interface permettant à l'utilisateur d'interagir avec la plateforme.

5.2.1 `index.html`

Le fichier `index.html` définit la structure des différents écrans de l'application : authentification, formulaire de demande d'accès, vérification MFA et affichage du résultat.

5.2.2 `style.css`

Le fichier `style.css` contient la mise en forme graphique de l'application et assure la présentation des différents écrans du workflow.

5.2.3 `config.js`

Le fichier `config.js` centralise les paramètres nécessaires au frontend, notamment les adresses de Keycloak, du backend et de l'agent local.

5.2.4 `app.js`

Le fichier `app.js` contient la logique principale exécutée dans le navigateur. Il assure notamment :

— l'intégration avec Keycloak ;
— l'envoi de la demande d'accès au backend ;
— la gestion des différentes étapes du workflow ;
— l'enrôlement et la vérification MFA ;
— la transmission du `launch_token` à l'agent local.

Le navigateur ne reçoit jamais directement les identifiants AWS temporaires.

5.3 Backend et orchestration du workflow

Le répertoire `backend` constitue le cœur de l'application. Il orchestre les différentes étapes du processus JIT sans intégrer directement les règles de décision.

5.3.1 `main.py`

Le fichier `main.py` constitue le point d'entrée de l'API FastAPI. Il coordonne le workflow selon les étapes suivantes :

1. réception de la demande d'accès ;
2. interrogation de Jira ;
3. construction du contexte et appel à OPA ;
4. mise en attente de la demande pour le MFA ;
5. vérification du code TOTP ;
6. appel à AWS STS ;
7. création du jeton destiné à l'agent local.

Il expose notamment les endpoints `/api/access-requests`, `/verify-mfa`, `/api/mfa/enroll` et `/api/agent/claim`.

5.3.2 `auth.py`

Le fichier `auth.py` assure la validation des jetons JWT émis par Keycloak. Il récupère les clés publiques JWKS de Keycloak et vérifie la signature du token Bearer transmis avec les requêtes protégées.

Ainsi, le backend n'a pas besoin de manipuler les mots de passe des utilisateurs.

5.3.3 `models.py`

Le fichier `models.py` définit les modèles Pydantic utilisés pour valider les données reçues par l'API, notamment la demande d'accès, le code MFA et les rôles temporaires pouvant être demandés.

5.3.4 `session_store.py`

Le fichier `session_store.py` assure le stockage temporaire des credentials AWS après leur génération par STS.

Au lieu de retourner les credentials au navigateur, il génère un `launch_token` aléatoire et à durée de vie courte. Ce jeton est ensuite utilisé une seule fois par l'agent local pour retirer les credentials.

5.4 Services d'intégration

Le dossier `backend/services` regroupe les modules permettant au backend de communiquer avec les différents services externes.

5.4.1 `jira_service.py`

Ce module communique avec l'API REST de Jira afin de récupérer les informations du ticket fourni dans la demande, notamment son statut, sa priorité et son assignation.

5.4.2 `opa_service.py`

Ce module transmet à Open Policy Agent le contexte complet de la demande et récupère la décision d'autorisation retournée par OPA.

5.4.3 `mfa_service.py`

Ce service assure la génération des secrets TOTP, la création de l'URI d'enrôlement et la vérification des codes MFA saisis par l'utilisateur.

5.4.4 `aws_service.py`

Le fichier `aws_service.py` assure le provisioning des accès AWS. Il associe le rôle demandé dans l'application au rôle IAM correspondant puis utilise **boto3** pour effectuer l'opération `AssumeRole` auprès d'AWS STS.

Il récupère ensuite les credentials temporaires ainsi que leur date d'expiration.

5.4.5 `splunk_service.py`

Ce service transmet les événements importants du workflow à Splunk à travers le HTTP Event Collector. Il permet notamment de journaliser les refus d'accès, les échecs MFA, les erreurs de provisioning et les accès accordés.

5.5 Politique d'autorisation OPA

Le répertoire `opa/policies` contient le fichier `jit_authz.rego`, qui représente la politique d'autorisation de la plateforme.

Cette politique définit notamment :

— la correspondance entre les rôles métier et les rôles temporaires autorisés ;
— la validité du ticket Jira ;
— l'assignation du ticket au demandeur ;
— la priorité minimale requise ;
— les limites de durée de la demande ;
— les contraintes temporelles.

La décision d'autorisation est donc externalisée du code FastAPI et gérée selon une approche *Policy as Code*.

5.6 Agent local

Le fichier `agent_jit.py` représente le composant installé sur le poste de travail de l'utilisateur.

L'agent expose localement un service HTTPS. Lorsqu'il reçoit un `launch_token`, il contacte directement le backend pour récupérer les credentials AWS temporaires.

Il injecte ensuite ces credentials dans une nouvelle session PowerShell à travers les variables d'environnement :

— `AWS_ACCESS_KEY_ID` ;
— `AWS_SECRET_ACCESS_KEY` ;
— `AWS_SESSION_TOKEN`.

Une session AWS CLI est ainsi disponible sans affichage ni saisie manuelle des credentials. L'agent surveille également leur date d'expiration afin de fermer automatiquement la session PowerShell correspondante.

5.7 Sécurisation des communications

Le répertoire `certs` contient le certificat `localhost.crt` et sa clé privée `localhost.key`. Ils sont utilisés pour activer HTTPS entre les composants locaux de l'application.

Le script `generate_local_cert.py` permet de générer ces certificats auto-signés pour l'environnement de développement.

Le fichier `serve_frontend_https.py` sert quant à lui l'interface Web sur HTTPS afin d'éviter l'utilisation de communications en clair.

5.8 Infrastructure Docker

Le fichier `docker-compose.yml` regroupe les services d'infrastructure nécessaires à l'application :

— **PostgreSQL**, utilisé comme base de données de Keycloak ;
— **Keycloak**, pour l'identité et l'authentification ;
— **Open Policy Agent**, pour la décision d'autorisation ;
— **Splunk**, pour la journalisation et la supervision.

Ces services sont connectés à travers le réseau Docker `jit-network`. L'utilisation de Docker Compose permet de démarrer et de reproduire facilement l'environnement nécessaire à la plateforme.

5.9 Scripts de démarrage

Plusieurs scripts PowerShell simplifient le lancement de l'environnement :

— `start_backend_https.ps1` démarre l'API FastAPI sur le port 8000 ;
— `start_frontend_https.ps1` démarre l'interface sur le port 5500 ;
— `start_agent.ps1` démarre l'agent local.

Ces scripts permettent de lancer les différentes parties de l'application avec leur configuration HTTPS sans devoir saisir manuellement les commandes associées.

5.10 Implémentation de l'application Just-In-Time Access

Cette section illustre le fonctionnement concret de la plateforme à travers le parcours réel d'un utilisateur, depuis la connexion jusqu'à l'utilisation effective des privilèges temporaires obtenus. Chaque étape de l'interface correspond directement à l'une des phases du workflow décrites en §2.3 et §4.3.2, le frontend matérialisant ainsi l'enchaînement *ticket* → *politique* → *MFA* → *accès* appliqué côté backend.

5.10.1 Page d'accueil et authentification

La page d'accueil de l'application présente le principe de fonctionnement de la plateforme et invite l'utilisateur à s'authentifier. Aucune action n'est possible avant cette étape : l'ensemble des routes de l'API backend étant protégé par la vérification du jeton Keycloak (voir §4.3.1), l'utilisateur est nécessairement redirigé vers l'écran de connexion Keycloak au clic sur le bouton dédié.

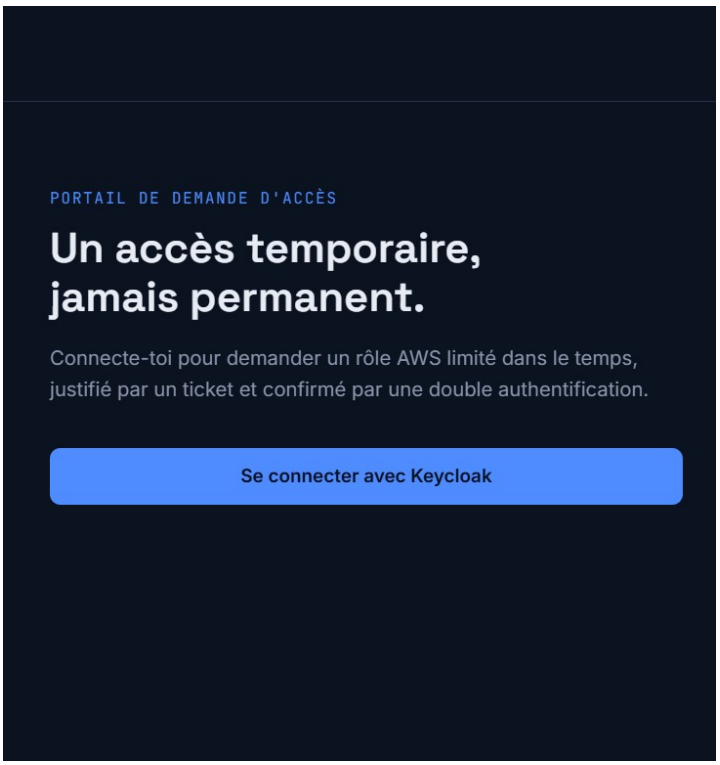


Figure 5.1 – Page d'accueil du portail de demande d'accès

Ce parcours correspond à l'initialisation du flow OpenID Connect *Authorization Code* avec PKCE décrit en §2.2 : le frontend délègue entièrement la saisie des identifiants à Keycloak et ne récupère, en retour, qu'un jeton d'accès signé.

5.10.2 Soumission d'une demande d'accès

Une fois authentifié, l'utilisateur accède au formulaire de demande. Celui-ci matérialise les quatre étapes du workflow — **TICKET**, **POLITIQUE**, **MFA** et **ACCÈS** — sous forme d'un indicateur de progression, permettant à l'utilisateur de visualiser l'étape en cours de traitement.

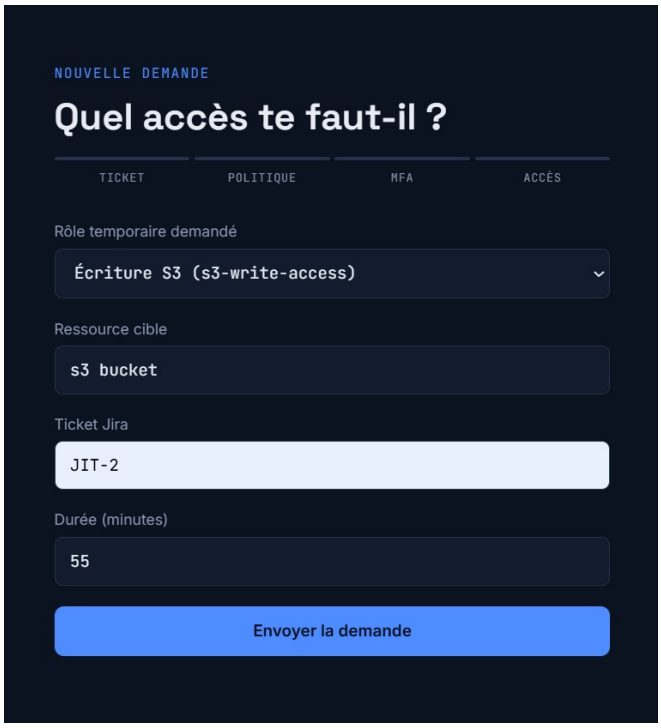


Figure 5.2 – Formulaire de soumission d'une demande d'accès temporaire

Le formulaire collecte les quatre informations strictement nécessaires à l'évaluation de la demande par OPA :

— le **rôle temporaire demandé** (ici `s3-write-access`), issu d'une liste fermée définie côté backend (`models.py`) ;
— la **ressource cible** concernée par ce rôle ;
— la **référence du ticket Jira** servant de justification (ici `JIT-2`) ;
— la **durée d'accès souhaitée**, exprimée en minutes.

À la soumission, ces informations sont transmises à l'endpoint `POST /api/access-requests`, qui déclenche côté backend la récupération du ticket auprès de Jira puis l'évaluation de la demande par OPA, comme détaillé en §4.3.2.

5.10.3 Refus d'une demande non conforme

Lorsque l'une des conditions imposées par la politique Rego n'est pas satisfaite — ticket non assigné au demandeur, priorité insuffisante, durée hors bornes autorisées, rôle

non compatible avec le profil de l'utilisateur, etc. — la demande est immédiatement rejetée, sans qu'aucune étape supplémentaire (MFA, provisioning AWS) ne soit engagée.



Figure 5.3 – Écran affiché lorsque la demande est refusée par la politique de sécurité

Ce comportement reflète directement la réponse renvoyée par le backend lorsque la variable `allowed` calculée par OPA vaut `false` :

Listing 5.1 – Réponse du backend en cas de refus

```
{
  "status": "denied",
  "reason": "Politique de securite non satisfaite"
}
```

Ce refus est également transmis à Splunk sous la forme d'un événement `access_denied`, afin d'en conserver la trace (voir §5.10.8).

5.10.4 Authentification multifacteur

Lorsque la demande est acceptée par OPA, l'utilisateur n'obtient pas directement ses privilèges : une étape de *step-up authentication* par code TOTP est imposée, conformément au principe présenté en §2.1.

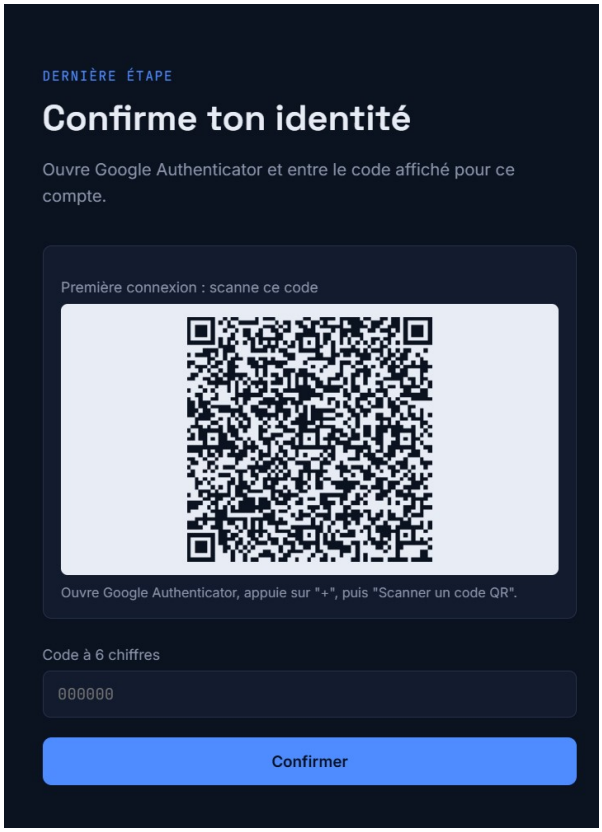


Figure 5.4 – Étape d'authentification multifacteur – enrôlement TOTP

Lors de la toute première demande d'un utilisateur, le backend (`mfa_service.py`) génère un secret TOTP unique et renvoie une URI de provisioning via l'endpoint `GET /api/mfa/enroll`. Cette URI est convertie côté frontend en QR code, que l'utilisateur scanne avec une application compatible (Google Authenticator ou Microsoft Authenticator). Un code à six chiffres, renouvelé toutes les trente secondes, doit ensuite être saisi et validé par le backend via l'endpoint `POST /api/access-requests/{request_id}/verify-mfa`. Pour les demandes suivantes, l'utilisateur étant déjà enrôlé, seule la saisie du code est requise — le QR code n'est plus régénéré.

5.10.5 Octroi de l'accès et lancement de la session

Une fois le code MFA validé, le backend assume le rôle IAM correspondant via AWS STS (`AssumeRole`) et génère des identifiants temporaires. Ces identifiants ne transitent **jamais** par le navigateur : seul un jeton opaque à usage unique (`launch_token`) est renvoyé au frontend, lequel le transmet à l'agent local installé sur le poste de l'utilisateur.



Figure 5.5 – Confirmation de l'octroi de l'accès et ouverture automatique de la session

L'agent local échange ce jeton contre les véritables identifiants AWS auprès de l'endpoint `POST /api/agent/claim`, un retrait qui ne peut être effectué qu'une seule fois :

Listing 5.2 – Retrait unique des identifiants par l'agent local

```
POST /api/agent/claim
Authorization: Bearer <launch_token>

-> { "credentials": { AccessKeyId, SecretAccessKey, SessionToken,
   expiration } }
```

L'interface affiche uniquement l'heure d'expiration de la session, sans jamais exposer les identifiants eux-mêmes, ce qui limite le risque de fuite par capture d'écran ou consultation ultérieure de l'historique du navigateur.

5.10.6 Session PowerShell et utilisation de l'AWS CLI

Dès réception des identifiants, l'agent local ouvre automatiquement une session PowerShell dans laquelle les trois variables d'environnement attendues par l'AWS CLI (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`) sont injectées.

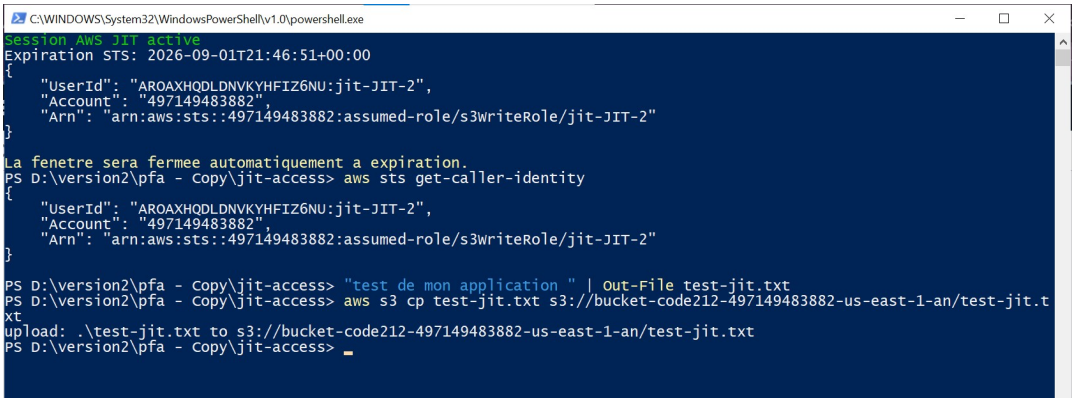


Figure 5.6 – Session PowerShell disposant des identifiants temporaires et exécution de commandes AWS CLI

La commande `aws sts get-caller-identity` confirme que la session est bien associée au rôle temporaire assumé, et non à une identité permanente. L'utilisateur peut ensuite exécuter des commandes AWS CLI, ici un envoi de fichier vers S3 (`aws s3 cp`), avec exactement les permissions accordées par le rôle temporaire — ni plus, ni moins — conformément au principe du moindre privilège.

5.10.7 Vérification du résultat côté AWS

Enfin, la console AWS permet de confirmer que l'opération réalisée depuis la session temporaire a bien été appliquée sur la ressource ciblée par la demande initiale.

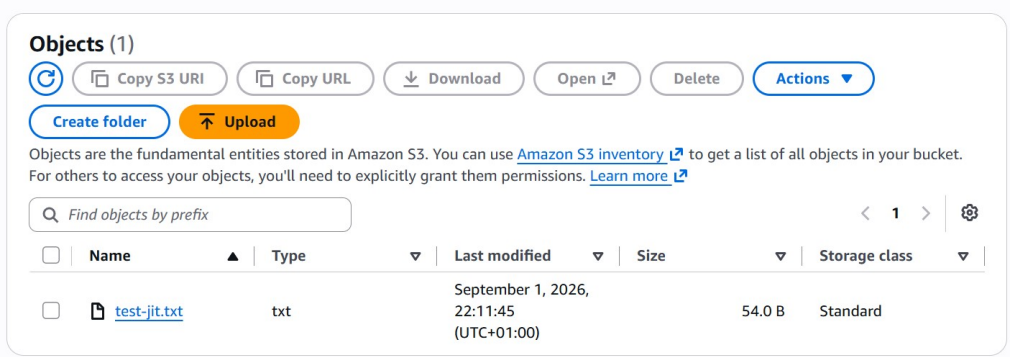


Figure 5.7 – Fichier déposé dans le bucket S3 grâce aux privilèges temporaires obtenus

Ce test de bout en bout valide l'ensemble de la chaîne décrite dans ce chapitre : une demande justifiée par un ticket Jira, évaluée par une politique Rego, confirmée par une authentification multifacteur, s'est traduite par un accès AWS réel, limité dans le temps et dans son périmètre d'action, sans qu'aucun identifiant permanent n'ait été communiqué à l'utilisateur.

5.10.8 Journalisation et supervision avec Splunk

Chaque événement contient systématiquement l'identifiant de la demande (`request_id`), l'adresse électronique de l'utilisateur et un horodatage précis, permettant de reconstituer l'intégralité du parcours d'une demande donnée en corrélant les événements par `request_id`.

Exemples d'événements collectés

Les captures suivantes illustrent des événements réels collectés lors des tests de la plateforme, consultables depuis l'interface de recherche de Splunk (`sourcetype=jit_access_logs`).

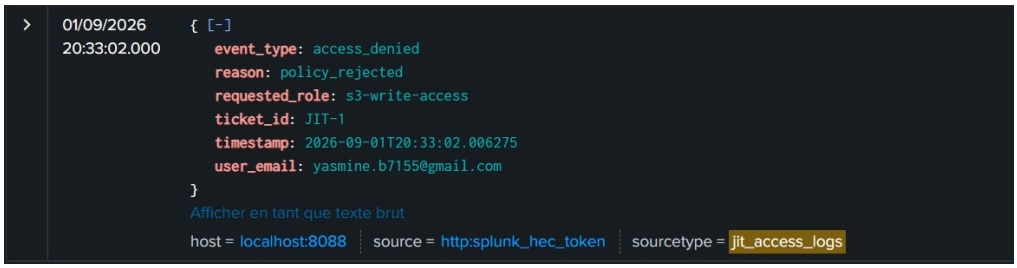


Figure 5.8 – Événement `access_denied` – demande rejetée par la politique OPA (ticket JIT-1)

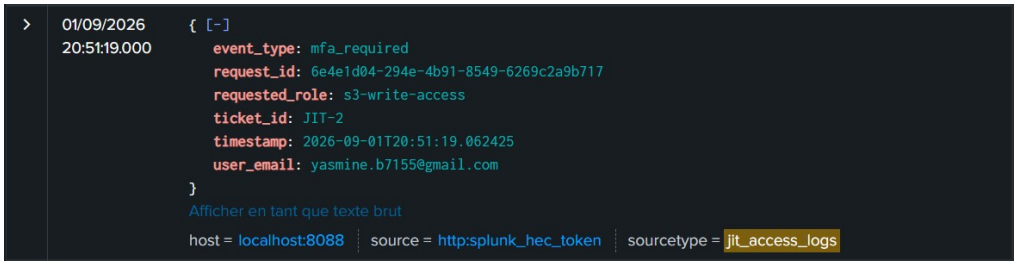


Figure 5.9 – Événement `mfa_required` – demande autorisée, en attente du code TOTP



Figure 5.10 – Événement `mfa_failed` – code TOTP invalide soumis par l'utilisateur

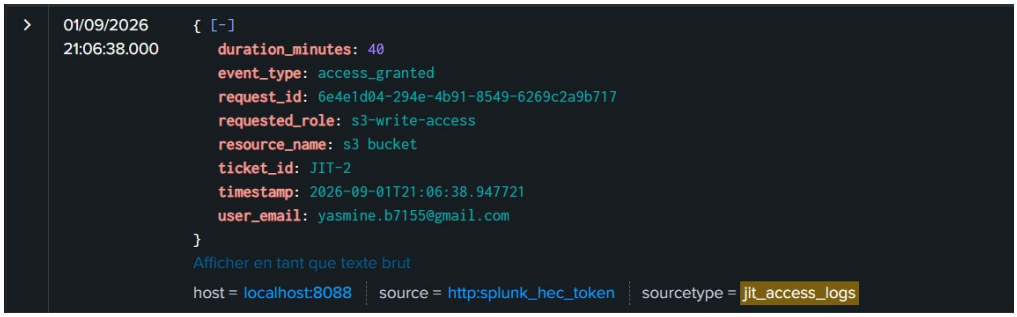


Figure 5.11 – Événement `access_granted` – provisioning AWS STS réussi (rôle `s3-write-access`, 40 minutes)

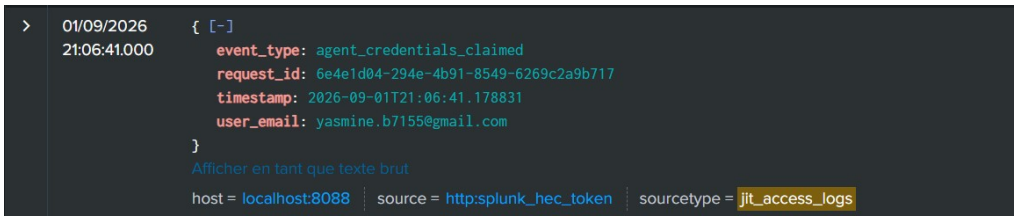


Figure 5.12 – Événement `agent_credentials_claimed` – retrait des identifiants par l'agent local

En rapprochant ces cinq événements par leur `request_id` commun, il est possible de reconstituer précisément la chronologie d'une demande ayant essuyé un premier échec MFA avant d'aboutir à un octroi effectif : décision favorable d'OPA à 20h51, échec de code TOTP à 21h06, validation réussie quelques secondes plus tard, provisioning AWS STS, puis récupération des identifiants par l'agent local. Cette continuité illustre concrètement l'intérêt de la piste d'audit présentée en §2.5 : chaque tentative, réussie ou non, reste visible et exploitable a posteriori.

Point clé. Splunk n'intervient à aucun moment dans la décision d'autoriser ou de refuser une demande : son rôle est strictement *a posteriori*, limité à la collecte, la centralisation et la mise à disposition des événements pour l'audit et la supervision. La logique de décision reste entièrement portée par OPA (§4.3.2).

Conclusion

Ce chapitre a présenté l'implémentation concrète de la plateforme de gestion des accès Just-In-Time, en complément des choix architecturaux et technologiques détaillés dans les chapitres précédents. L'organisation modulaire du projet — séparant clairement l'interface utilisateur, l'orchestration du workflow assurée par le backend FastAPI, les services d'intégration dédiés à chaque composant externe (Keycloak, Jira, OPA, AWS, Splunk), la politique d'autorisation Rego et l'agent local — traduit directement, au

niveau du code, la séparation des responsabilités défendue sur le plan théorique au chapitre 2 : chaque brique reste indépendante, remplaçable et testable isolément, sans logique de décision codée en dur dans l'application.

Le parcours utilisateur illustré à travers les captures d'écran de la section 5.10 a permis de valider ce fonctionnement de bout en bout sur un cas réel : authentification via Keycloak, soumission d'une demande justifiée par un ticket Jira, évaluation par la politique OPA — avec démonstration des deux issues possibles, refus et autorisation —, validation d'un second facteur TOTP, provisioning d'identifiants AWS temporaires via STS, distribution sécurisée par l'agent local jusqu'à une session PowerShell dédiée, et enfin exécution effective d'une opération S3 avec les seules permissions accordées. Les événements collectés dans Splunk ont par ailleurs confirmé qu'il est possible de reconstituer, a posteriori et par simple corrélation d'un identifiant de demande, l'intégralité de cette chronologie — y compris un échec initial d'authentification multifacteur suivi d'une tentative réussie.

Cette implémentation démontre ainsi que les principes présentés en amont — *Zero Standing Privilege*, moindre privilège, *policy-as-code*, *fail-closed* par défaut et non-interférence de la couche d'observabilité — ne sont pas restés de simples notions théoriques, mais ont été effectivement traduits en un système fonctionnel, cohérent et vérifiable. Le chapitre suivant, qui clôt ce rapport, revient sur l'ensemble de la démarche menée durant ce stage, dresse un bilan par rapport aux objectifs initiaux et propose des perspectives d'amélioration pour rapprocher davantage cette implémentation d'une solution prête pour un environnement de production.

Conclusion générale

Synthèse des travaux réalisés

Ce Projet de Fin d'Année, réalisé au sein du Centre Code212 de l'Université Cadi Ayyad, avait pour objectif de concevoir et d'implémenter une plateforme de gestion des accès privilégiés *Just-In-Time*, permettant d'accorder des privilèges Cloud uniquement lorsqu'ils sont nécessaires, pour une durée limitée, et après vérification de la légitimité de la demande.

La démarche adoptée a permis de couvrir l'ensemble du cycle de vie d'un accès temporaire. Le cadre théorique a d'abord posé les bases conceptuelles du projet — *Zero Standing Privilege*, principe du moindre privilège, modèles RBAC et ABAC, *policy-as-code* — avant que le choix des technologies ne traduise ces principes en composants concrets : Keycloak pour l'authentification, Jira pour la justification métier, Open Policy Agent pour la décision d'autorisation, TOTP pour l'authentification renforcée, AWS IAM et STS pour la génération d'identifiants temporaires, un agent local pour leur distribution sécurisée, et Splunk pour la traçabilité.

L'implémentation réalisée a permis de valider ce fonctionnement de bout en bout, depuis la création d'un ticket Jira jusqu'à l'exécution effective de commandes AWS CLI dans une session PowerShell temporaire, en passant par l'évaluation d'une politique Rego, la validation d'un code TOTP et le provisioning d'un rôle IAM via `AssumeRole`. Les captures d'écran présentées au chapitre précédent, tant côté application que côté Splunk et console AWS, démontrent que l'ensemble de la chaîne fonctionne de manière cohérente et que chaque décision — octroi ou refus — reste tracée et justifiable.

Bilan par rapport aux objectifs initiaux

Les enjeux identifiés en introduction ont été largement adressés par la solution développée. La réduction des privilèges permanents est effective : aucun utilisateur ne dispose de droits AWS en dehors d'une session temporaire explicitement accordée. La légitimité de chaque demande est vérifiée à travers le ticket Jira et la politique Rego, qui centralise l'ensemble des règles d'autorisation en dehors du code applicatif. L'authentification est renforcée par une étape TOTP indépendante de la session Keycloak, et la distribution des identifiants évite toute exposition directe dans le navigateur. Enfin, la traçabilité est assurée par Splunk, chaque événement — refus, échec MFA, octroi, retrait des identifiants — restant consultable et corrélable par identifiant de demande.

Le projet a ainsi permis de démontrer, à l'échelle d'une implémentation fonctionnelle,

qu'il est possible de composer des briques open source et des services Cloud standards pour aboutir à un système appliquant rigoureusement le principe de *Zero Standing Privilege*, sans dépendre d'une solution PAM commerciale fermée.

Limites de la solution actuelle

Certaines limites, assumées dans le périmètre défini au premier chapitre, doivent être soulignées afin de situer correctement la portée de ce travail. Le stockage des demandes en attente de MFA et des secrets TOTP repose actuellement sur des structures en mémoire côté backend, adaptées à une démonstration mais non persistantes ni adaptées à un déploiement multi-instance. La solution ne couvre qu'un seul fournisseur Cloud (AWS) et qu'un seul type de ressource (Amazon S3), les certificats TLS utilisés restent auto-signés et propres à un environnement de développement local, et aucune limitation du nombre de tentatives de code TOTP n'a été mise en place. Ces choix, cohérents avec la durée du stage et la priorité donnée à la validation du principe architectural, définissent naturellement les axes d'amélioration présentés ci-après.

Perspectives

Plusieurs pistes d'évolution permettraient de rapprocher davantage cette implémentation d'une solution de production.

— **Alertes Splunk sur les échecs répétés de MFA.** Les événements `mfa_failed` sont aujourd'hui journalisés mais ne déclenchent aucune action automatique. La mise en place d'une alerte Splunk se déclenchant lorsqu'un même utilisateur — ou une même adresse IP — accumule plusieurs échecs de code TOTP sur une courte période permettrait de détecter une éventuelle tentative de contournement du second facteur et de notifier un administrateur en temps réel, plutôt que de se limiter à une consultation a posteriori des journaux.

— **Limitation du nombre de tentatives MFA (rate limiting).** En complément de l'alerte précédente, un blocage temporaire après un nombre défini d'échecs consécutifs renforcerait la résistance de l'étape TOTP face à une attaque par force brute sur le code à six chiffres.

— **Révocation anticipée des sessions AWS.** La solution actuelle laisse expirer les identifiants temporaires uniquement à l'échéance initialement demandée. Un mécanisme permettant à un administrateur de révoquer immédiatement une session en cours — par exemple via une politique IAM de révocation ou l'invalidation du `launch_token` associé — ajouterait un contrôle réactif en cas d'usage suspect détecté pendant la session.

— **Persistance des données en base de données.** Le remplacement des dictionnaires en mémoire (`PENDING_REQUESTS`, `USER_TOTP_SECRETS`) par une base de données persistante permettrait de conserver un historique complet des demandes, de survivre à un redémarrage du backend et de préparer un déploiement à plusieurs instances.
— **Extension à d'autres ressources et fournisseurs Cloud.** Le modèle de rôles temporaires pourrait être étendu à d'autres services AWS (EC2, RDS, Lambda) ainsi qu'à d'autres fournisseurs Cloud (Azure, GCP), en généralisant la correspondance rôle métier / rôle temporaire actuellement limitée à S3.
— **Durée d'accès adaptative selon le niveau de risque.** La durée maximale actuellement fixe (240 minutes) pourrait être modulée dynamiquement par la politique Rego en fonction d'attributs contextuels supplémentaires, tels que la sensibilité de la ressource demandée ou l'historique récent de l'utilisateur.
— **Infrastructure à clés publiques de production.** Le remplacement des certificats auto-signés par des certificats émis par une autorité de certification interne ou publique serait nécessaire avant tout déploiement au-delà d'un environnement de test local.
— **Tests automatisés et intégration continue.** La mise en place de tests unitaires sur les politiques Rego (via `opa test`) et sur les endpoints FastAPI, associée à une chaîne d'intégration continue, garantirait la non-régression du système à mesure que de nouvelles règles ou fonctionnalités seraient ajoutées.
— **Interface d'administration et tableaux de bord.** Un tableau de bord dédié, s'appuyant sur les données déjà collectées par Splunk, permettrait à un administrateur de visualiser en un coup d'œil les demandes en cours, l'historique des octrois et refus, ainsi que les utilisateurs les plus actifs.

En définitive, ce projet a permis de concevoir, d'implémenter et de valider une architecture complète de gestion des accès privilégiés fondée sur le principe de *Zero Standing Privilege*. Au-delà du résultat technique obtenu, il a constitué une occasion concrète d'approfondir des notions essentielles de la cybersécurité moderne — gestion des identités, contrôle d'accès dynamique, authentification renforcée, délégation d'identité Cloud et observabilité — et de mesurer, à travers une implémentation réelle, la manière dont ces concepts s'articulent pour réduire durablement la surface d'attaque liée aux privilèges permanents.

Bibliographie

[1] V. C. Hu et al., *SP 800-162 : Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, National Institute of Standards and Technology, 2014.

[2] D. Hardt (éd.), *The OAuth 2.0 Authorization Framework*, RFC 6749, Internet Engineering Task Force (IETF), 2012.

[3] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, C. Mortimore, *OpenID Connect Core 1.0*, OpenID Foundation, 2014.

[4] N. Sakimura, J. Bradley, N. Agarwal, *Proof Key for Code Exchange by OAuth Public Clients*, RFC 7636, IETF, 2015.

[5] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.

[6] M. Jones, *JSON Web Key (JWK)*, RFC 7517, IETF, 2015.

[7] D. M'Raihi, S. Machani, M. Pei, J. Rydell, *TOTP : Time-Based One-Time Password Algorithm*, RFC 6238, IETF, 2011.

[8] Keycloak, *Keycloak Documentation*, `https://www.keycloak.org/documentation`.

[9] Keycloak, *Keycloak JavaScript Adapter*, `https://www.keycloak.org/securing-apps/javascript-adapter`.

[10] Open Policy Agent, *OPA Documentation*, `https://www.openpolicyagent.org/docs/latest/`.

[11] Open Policy Agent, *Policy Language – Rego*, `https://www.openpolicyagent.org/docs/latest/policy-language/`.

[12] Atlassian, *Jira Cloud platform REST API documentation*, `https://developer.atlassian.com/cloud/jira/platform/rest/v3/`.

[13] Amazon Web Services, *AWS Identity and Access Management (IAM) Documentation*, `https://docs.aws.amazon.com/IAM/`.

[14] Amazon Web Services, *AWS Security Token Service (STS) Documentation*, `https://docs.aws.amazon.com/STS/`.

[15] Amazon Web Services, *AssumeRole – AWS Security Token Service API Reference*, `https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html`.

[16] Amazon Web Services, *Boto3 – The AWS SDK for Python Documentation*, `https://boto3.amazonaws.com/v1/documentation/api/latest/index.html`.

[17] Amazon Web Services, *Amazon Simple Storage Service (S3) Documentation*, https://docs.aws.amazon.com/s3/.

[18] Splunk Inc., *Splunk HTTP Event Collector (HEC) Documentation*, https://docs.splunk.com/Documentation/Splunk/latest/Data/UsetheHTTPEventCollector.

[19] Docker Inc., *Docker Documentation*, https://docs.docker.com/.

[20] Docker Inc., *Docker Compose Documentation*, https://docs.docker.com/compose/.

[21] S. Ramírez, *FastAPI Documentation*, https://fastapi.tiangolo.com/.

[22] Pydantic, *Pydantic Documentation*, https://docs.pydantic.dev/.

[23] PyOTP, *PyOTP – Python One-Time Password Library*, https://pyauth.github.io/pyotp/.

[24] Encode, *HTTPX Documentation*, https://www.python-httpx.org/.

[25] Amazon Web Services, *AWS Command Line Interface (CLI) Documentation*, https://docs.aws.amazon.com/cli/.

[26] Amazon Web Services, *AWS IAM Identity Center Documentation*, https://docs.aws.amazon.com/singlesignon/.